

Physics-informed neural networks and the Deep Ritz method for diffusion, wave and Laplace problems: a reproducible benchmark and its failure modes

Xiaoxiao Zhouyi

School of Engineering Mathematics and Technology, University of Bristol, UK.

zhouyixiaoxiao@gmail.com

Abstract

We benchmark physics-informed neural networks (PINNs) and the Deep Ritz method on eight linear problems (dimension $d \leq 20$) with exact solutions proved unique in a stated class, using seeded paired runs and low-dimensional classical baselines. An L-BFGS phase after Adam lowers the error in all 175 pairs; the largest collocation penalty after it (cell-centred grid, heat problem) comes from missing points near the initial line. A small unit-weight PINN fails on a two-mode wave problem; finite differences are over 400 times faster at equal or better accuracy. The PINN is more accurate per iteration up to $d = 10$, Deep Ritz at equal CPU time at $d = 10$. On the unit cube we prove that the harmonic-extension norm from the boundary has exact order $d^{-1/4}$; with it, as already with Payne’s constant, the two population PINN loss terms bound the L^2 error of any C^2 network. The bound held on all 133 networks, exceeding the error 1.9 to 3.4 times at $d \geq 10$, up to 25 at $d = 2$, missing a pre-registered target (≤ 3 for $d \geq 5$) on 21 of 57 networks. At $d = 20$ Legendre input features (containing the Poisson solution to 0.4 per cent), centred inputs (exploratory) and, for Deep Ritz, a featureless reparametrisation each remove a plateau left by a fixed affine part. The features passed a pre-registered generality test on four problems but, like the affine part, hurt on a fifth problem, added afterwards (not blind). Minimal examples show two pitfalls: a residual zeroed by misused automatic differentiation, and too few boundary conditions.

Keywords: physics-informed neural networks; Deep Ritz method; collocation sampling; L-BFGS; finite-difference baselines; high-dimensional Poisson equation; a posteriori error bound; biharmonic Steklov eigenvalue; reproducibility.

1 Introduction

A physics-informed neural network (PINN) represents the solution of a partial differential equation by a neural network whose weights minimise the squared residual of the equation at collocation points, plus the misfit to the initial and boundary data [47]; neural trial functions for differential equations go back at least to Lagaris et al. [28]. The Deep Ritz method [12] minimises instead a Monte-Carlo estimate of the variational energy of an elliptic problem, with the boundary condition imposed by a penalty. Both methods need no mesh, only automatic differentiation; they are short to implement [34] and extend formally to inverse problems [47] and to high dimension [53], where other neural solvers exist as well [20]; see the reviews [10, 23].

Their difficulties are equally well documented. The terms of a PINN loss can be badly unbalanced [58]; a neural-tangent-kernel analysis shows that they are then learnt at different rates and exhibits a two-mode wave problem on which the unweighted method fails [60], which we reuse as problem *Wave1*. PINNs that succeed on simple linear equations fail as the coefficients grow, for reasons of optimisation rather than expressivity [26], and time-dependent problems may need a loss that respects causality [61]. Quasi-Newton optimisation has been part of the

method since Raissi et al. [47]; Rathore et al. [48] measure the benefit of running Adam and then L-BFGS and relate it to the ill-conditioning of the loss. The placement of collocation points has been compared systematically by Wu et al. [63], with residual-based refinement and importance sampling as adaptive variants [34, 40]; Daw et al. [11] attribute failures of PINNs to a failed propagation of the solution from the initial and boundary points into the interior. Against classical solvers, PINNs did not beat the finite element method in solution time and accuracy in the study of Grossmann et al. [19]; McGreivy and Hakim [35] show that weak baselines and selective reporting have made machine-learning solvers for fluid-related equations look better than they are; and much larger benchmarks than ours exist [21]. On the theoretical side, PINN minimisers converge for linear elliptic and parabolic problems as the number of collocation points grows [50], and a boundary penalty is known to change the problem being solved [3, 39].

This article asks what the two methods deliver on the diffusion, wave and Laplace equations when every error is measured against an exact solution, every run is seeded, comparisons are paired and a classical solver is run alongside, and it contributes the following.

- (C1) Eight test problems, each stated so that it has a unique solution in a stated class: the unique classical solution or, for the three-dimensional problem with discontinuous data, the unique solution that attains the datum in mean square. The exact solutions are given with proofs (Section 2; Supplementary Sections S1 and S7).
- (C2) A seeded benchmark of 175 networks on five low-dimensional problems, with five collocation strategies and two optimiser arms of equal length, reporting held-out relative errors; for the heat equation the error is also resolved in time, which shows how a small space–time error hides a large error at late times (Sections 3 and 4).
- (C3) The effect of replacing the second half of the Adam iterations by L-BFGS, measured pair by pair and also against the best logged Adam iterate (Section 4.2).
- (C4) A paired comparison of the collocation strategies, with a control experiment that traces the one large penalty after the L-BFGS phase, that of a cell-centred grid on the heat problem, to the absence of residual points next to the initial line (Section 4.3).
- (C5) Longer runs and a compatible-data control on the two- and three-dimensional problems, which show what limits them; runs of up to 41 000 iterations showing that the two-mode wave problem is not solved by the unit-weight PINN at the network size used, with the trained networks projected onto the two modes; and a comparison of all five problems with second-order finite differences in accuracy and time (Section 5).
- (C6) A comparison of the PINN and the Deep Ritz method on the unit cube in dimension $d = 2$ to 20, in accuracy per iteration, in cost per iteration, at equal compute and at two budgets, with the penalty bias of Deep Ritz computed exactly (Sections 6.1–6.6).
- (C7) On the unit cube in any dimension d , two-sided bounds of exact order $d^{-1/2}$ for the squared norm of the harmonic extension from $L^2(\partial\Omega)$, with surface measure, to $L^2(\Omega)$, which is the inverse of the first biharmonic Steklov eigenvalue. With them the L^2 error of any network is bounded by the two terms of the PINN loss with a constant explicit in d , and the penalty bias of Deep Ritz by a sharper constant. The bound is evaluated under a pre-registered protocol on the trained networks of the dimension study and, without pre-registration, on networks retrained with separately written code (Section 6.7).
- (C8) A pre-registered test, for both methods, at equal iterations and at equal CPU time, with two pre-registered controls for Deep Ritz (lift3u, rep3) and an exploratory centred-input arm for both methods, of two cheap remedies for the plateau at $d = 20$, where neither method improves on the best affine approximation of the solution: a fixed affine part computed by one linear solve, and coordinate-wise Legendre input features (Section 6.8).
- (C9) Two implementation pitfalls demonstrated on minimal, released examples: a PDE residual made identically zero by differentiating with respect to a slice of the input and replacing the missing derivative by zeros, and a boundary-value problem with too few conditions, on which training reaches a small loss with a different function for every seed (Section 7).

No new method is proposed, and, except for the two-sided bound of (C7), none of these findings is new taken singly. The gain from an L-BFGS phase after Adam was measured by Rathore et al. [48]; what is added here is the same measurement, paired by seed, on the heat and Laplace problems at a small budget and against an untuned Adam (Section 4.2). That the point set matters mainly when points are scarce is in the direction reported by Wu et al. [63]; at our much smaller budgets we do not reproduce their ranking of grid and resampled points on every problem. That a PINN needs residual points next to the initial data (C4) is consistent with the propagation hypothesis of Daw et al. [11] and with causal training [61]; the control experiment isolates it for one problem. The late-time behaviour of the heat problem (C2) is an elementary consequence of a loss made of absolute residuals; we have not found it quantified for this problem and do not claim it as a new phenomenon. The cost gap to classical solvers agrees with Grossmann et al. [19], and the penalty bias is classical [3, 39]. The residual loss and the Ritz loss have been compared on elliptic problems by Chen et al. [8], who found, as we do, that neither dominates the other. The constant of (C7) is classical: it is $\delta_1^{-1/2}$ for the first biharmonic Steklov eigenvalue δ_1 , Payne’s bound for convex domains [44], as stated in Bucur and Gazzola [5, Lemma 5.3], already gives a constant independent of d , and the torsion function of the cube gives one that decreases slowly with d ; to our knowledge, what is new is the explicit two-sided bound of the squared constant $1/\delta_1$ of exact order $d^{-1/2}$ on the cube (equivalently, the constant itself is of exact order $d^{-1/4}$). The error bound that follows is an application of known theory with a better constant: a bound of the same form, on smooth domains and with a constant that is not explicit, is in Shin et al. [51, App. B]. The remedies of (C8) are known [1, 7, 28, 65]; only their measurement is added. Apart from (C7), what this article contributes is the setting: exact solutions, seeds, paired comparisons and a classical baseline of the kind whose absence McGreivy and Hakim [35] criticise; a verification of the benchmark and of the dimension study by re-aggregation of the raw runs and by separately written code for part of them (Section 3.9 states what it covers and what it does not); and the release of all code and data.

The scope is narrow. All problems are linear with constant coefficients on boxes; the networks are small and use the tanh activation; the loss weights are equal to one in Sections 4 and 5, and a single boundary weight per problem and method is chosen once in Section 6. Of the published remedies only two cheap ones were tried, at $d = 20$; adaptive loss weights, causal training, Fourier features and boundary conditions built into the trial function were not (Section 8.4). Each of these might change a conclusion, in particular the outcome on Wave1: on a variant with a faster second mode, Rathore et al. [48] report a few per cent with a larger network and budget and tuned settings, without any of these remedies (Section 5.3).

Section 2 states the problems and Section 3 the methods. Sections 4, 5 and 6 report the low-dimensional benchmark, the harder problems with the finite-difference baselines, and the dimension study, including the stability constant with the computable error bound (Section 6.7) and the two remedies (Section 6.8). Section 7 demonstrates the two implementation pitfalls, and Section 8 discusses the evidence and its limitations. The Supplementary Material contains the proofs (Section S7), further results, figures and tables for each section, and implementation details with full tables (Section S8); its sections, equations, figures and tables carry the prefix S.

2 Test problems and exact solutions

We use eight linear, constant-coefficient problems on boxes. Each has an exact solution in closed form or, in one case, as a series, so every error reported below is measured against the true solution and not against another numerical method. Five problems are low-dimensional (Heat, Wave1, Wave2, Lap2, Lap3), one is a control variant of Lap3 with compatible data (Lap3s), and two are posed on the unit cube in any dimension $d \geq 2$ (LapD, PoiD). Table 1 lists them with their origin and the held-out evaluation set; the networks are in Table 2. In the evolution problems t is the first coordinate and Δ acts on the space variables only. The evaluation sets are dense

grids, or random points drawn from a generator of their own, and are separate from the training points except for a few grid nodes on the boundary and, for **Lap3**, the points of one collocation strategy (Supplementary Section S1).

Heat equation. The solution of **Heat** decays to $e^{-\pi^2} = 5.17 \times 10^{-5}$ of its initial size at $t = 1$. Since $\|u^*(t, \cdot)\|_{L^2(0,1)} = e^{-\pi^2 t}/\sqrt{2}$, a given L^2 error on the time slice t is a relative error $\sqrt{2}e^{\pi^2 t}$ times as large on that slice, a factor $e^{\pi^2} \approx 1.93 \times 10^4$ larger at $t = 1$ than at $t = 0$. A small space–time relative error ε_2 therefore says little about late times, and we also report the relative error $\varepsilon_2^{\text{slice}}(t)$ on single time slices (both are defined in (10); results in Section 4.5).

Wave equations. Problem **Wave1** is the benchmark of Wang et al. [60, Sec. 7.3]:

$$u_{tt} = 4u_{xx}, \quad u(t, 0) = u(t, 1) = 0, \quad u(0, x) = \sin \pi x + \frac{1}{2} \sin 4\pi x, \quad u_t(0, x) = 0. \quad (1)$$

It superposes two standing modes with amplitudes 1 and $\frac{1}{2}$ and frequencies in the ratio 1 : 4: on $0 \leq t \leq 1$ the first completes one period and the second four. Problem **Wave2** is the two-dimensional wave equation

$$u_{tt} = \Delta u \text{ in } (-1, 1)^2, \quad u = 0 \text{ on } \partial\Omega, \quad u(0, x, y) = \sin \pi x \sin \pi y, \quad u_t(0, x, y) = 0. \quad (2)$$

Its solution is a single standing mode of period $\sqrt{2}$, so $0 \leq t \leq 1$ covers $1/\sqrt{2} \approx 0.71$ of one period; the solution changes sign at $t = 1/(2\sqrt{2}) \approx 0.35$ and its amplitude factor at $t = 1$ is $\cos(\sqrt{2}\pi) = -0.266$. A function that only decays in time is therefore far from the solution.

Laplace problems. Problem **Lap2** (Exercise 3 of Section 6.2 of Strauss [55]) mixes Dirichlet data on two sides with homogeneous Neumann data on the other two. The data are compatible at the four corners, because both Dirichlet data have zero y -derivative at $y = 0$ and $y = \pi$, and the solution is smooth up to the boundary. Problem **Lap3**,

$$\Delta u = 0 \text{ in } \Omega = (0, \pi)^3; \quad u(\pi, y, z) = \sin y \cos z, \quad u = 0 \text{ on the other five faces}, \quad (3)$$

is solved by separation of variables:

$$u^*(x, y, z) = \sin y \sum_{n \text{ even}} b_n \sin(nz) \frac{\sinh(k_n x)}{\sinh(k_n \pi)}, \quad k_n = \sqrt{1 + n^2}, \quad b_n = \frac{4n}{\pi(n^2 - 1)}, \quad (4)$$

where the sum runs over $n = 2, 4, 6, \dots$ and the b_n are the sine coefficients of $\cos z$ on $(0, \pi)$. Because $\cos z$ does not vanish at $z = 0$ and $z = \pi$, the data jump by $\sin y$ across the edges $\{x = \pi, z = 0\}$ and $\{x = \pi, z = \pi\}$. Two consequences matter numerically. A network is continuous on the closed cube, so it cannot match these boundary data to better than $\frac{1}{2}$ in the maximum norm, whatever its size; and u^* has infinite Dirichlet energy (Proposition S1(b)), so $u^* \notin H^1(\Omega)$ and a relative H^1 error is not defined. Errors for **Lap3** are therefore measured at the centres of a 40^3 grid of cells, which lie at least $\pi/80$ from every face. Problem **Lap3s** keeps everything except the datum on $x = \pi$, which becomes the compatible single mode $\sin y \sin z$; its solution is one separable mode instead of a slowly converging series. It therefore changes two things at once, the compatibility of the data and the modal content of the solution, and Section 5 compares the two problems with this in mind.

Problems in d dimensions. On $\Omega = (0, 1)^d$ with Dirichlet data taken from the exact solution we consider

$$\text{LapD : } \quad -\Delta u = 0 \text{ in } \Omega, \quad u = u^* \text{ on } \partial\Omega; \quad u^*(\mathbf{x}) = \sum_{k=1}^{\lfloor d/2 \rfloor} x_{2k-1} x_{2k}, \quad (5)$$

$$\text{PoiD : } \quad -\Delta u = \pi^2 u^* \text{ in } \Omega, \quad u = u^* \text{ on } \partial\Omega; \quad u^*(\mathbf{x}) = d^{-1/2} \sum_{i=1}^d \cos \pi x_i. \quad (6)$$

Table 1: The eight test problems. Evaluation set: the held-out points on which all errors are computed; grids are uniform and include the boundary unless they are cell-centred. In LapD and PoiD the Dirichlet datum is the restriction of u^* to $\partial\Omega$. The networks are listed in Table 2.

	Equation, domain	Data	Exact solution u^*	Origin	Evaluation set
Heat	$u_t = u_{xx};$ $x \in (0, 1),$ $t \in (0, 1]$	$u(0, x) = \sin \pi x;$ $u(t, 0) = u(t, 1) = 0$	$e^{-\pi^2 t} \sin \pi x$	standard single mode	201×201 grid in (t, x)
Wave1	$u_{tt} = 4u_{xx};$ $x \in (0, 1),$ $t \in (0, 1]$	$u(0, x) = \sin \pi x$ $+ \frac{1}{2} \sin 4\pi x;$ $u_t(0, x) = 0;$ $u(t, 0) = u(t, 1) = 0$	$\sin \pi x \cos 2\pi t$ $+ \frac{1}{2} \sin 4\pi x \cos 8\pi t$	Wang et al. [60], Sec. 7.3	201×201 grid in (t, x)
Wave2	$u_{tt} = \Delta u;$ $\Omega = (-1, 1)^2,$ $t \in (0, 1]$	$u(0, \cdot) = \sin \pi x \sin \pi y;$ $u_t(0, \cdot) = 0; u = 0$ on $\partial\Omega$	$\sin \pi x \sin \pi y \cos(\sqrt{2} \pi t)$	standard standing mode	$21 \times 81 \times 81$ grid in (t, x, y)
Lap2	$\Delta u = 0;$ $\Omega = (0, \pi)^2$	$u(0, y) = 0;$ $u(\pi, y) = \frac{1}{2}(1 + \cos 2y);$ $u_y(x, 0) = u_y(x, \pi) = 0$	$\frac{x}{2\pi} + \frac{\cos 2y \sinh 2x}{2 \sinh 2\pi}$	Strauss [55], Sec. 6.2, Ex. 3	201×201 grid
Lap3	$\Delta u = 0;$ $\Omega = (0, \pi)^3$	$u(\pi, y, z) = \sin y \cos z;$ $u = 0$ on the other five faces	series (4)	the setting of Strauss [55], Sec. 6.2	40^3 cell centres
Lap3s	$\Delta u = 0;$ $\Omega = (0, \pi)^3$	$u(\pi, y, z) = \sin y \sin z;$ $u = 0$ on the other five faces	$\sin y \sin z \frac{\sinh(\sqrt{2} x)}{\sinh(\sqrt{2} \pi)}$	control variant of Lap3	40^3 cell centres
LapD	$\Delta u = 0;$ $\Omega = (0, 1)^d$	$u = u^*$ on $\partial\Omega$	$\sum_{k=1}^{\lfloor d/2 \rfloor} x_{2k-1} x_{2k}$	E and Yu [12], Sec. 3.2 ($d = 10$)	50 000 uniform random points
PoiD	$-\Delta u = \pi^2 u^*;$ $\Omega = (0, 1)^d$	$u = u^*$ on $\partial\Omega$	$d^{-1/2} \sum_{i=1}^d \cos \pi x_i$	solution of E and Yu [12], Sec. 3.3 (a Neumann problem, $d = 5, 10$), rescaled; posed here as a Dirichlet problem	50 000 uniform random points

At $d = 10$, LapD is the test problem of E and Yu [12, Sec. 3.2]; for odd d the last coordinate does not enter. The solution of PoiD is, up to the factor $d^{-1/2}$, that of the Neumann example of E and Yu [12, Sec. 3.3], who solve $-\Delta u + \pi^2 u = 2\pi^2 \sum_k \cos \pi x_k$ with zero normal derivative for $d = 5$ and 10 by the Ritz method without a boundary penalty; here the same function is the solution of a Dirichlet problem for the Poisson equation, so that both methods can be applied to it with a penalty. With $m = \lfloor d/2 \rfloor$ the values of the solution of LapD have mean $m/4$ and standard deviation $\sqrt{7m}/12$, so they concentrate around the mean as d grows and a small relative L^2 error becomes cheap: by Proposition 1 the best constant has $\varepsilon_2 = 0.661$ at $d = 2$ but 0.269 at $d = 20$, and the best affine function 0.250 and 0.102. PoiD is normalised against this effect: it has zero mean and $\|u^*\|_{L^2(\Omega)}^2 = \frac{1}{2}$ for every d , so the constant floor is 1 and the affine floor 0.1203 in every dimension. Its normal derivative, a multiple of $\sin \pi x_i$ on the faces $x_i \in \{0, 1\}$, vanishes on every face, which makes it the favourable case for the penalised Deep Ritz energy (Proposition 2); LapD, whose normal derivative does not vanish, is the representative one.

Existence, uniqueness and reference levels. By a classical solution we mean one that is twice continuously differentiable on the closed domain (the closed space–time cylinder for the evolution problems). Proposition S1 in the Supplementary Material shows that each u^* of Table 1 is a classical solution, except for Lap3, whose series solution is smooth up to the five faces with

zero data and attains the datum on the sixth face in mean square; Proposition S2 shows that each problem has no other solution in that class. Continuous dependence on the data, the remaining requirement for well-posedness, is classical for linear problems of these types [14] and is not re-proved here. The formulas were also checked symbolically and numerically (Supplementary Section S1). The last statement gives the levels that a trained network must beat before its error means anything:

Proposition 1 (Trivial-predictor floors). *Let $\Omega = (0, 1)^d$ with $d \geq 2$, let $m = \lfloor d/2 \rfloor$, and let $\|\cdot\|$ be the norm of $L^2(\Omega)$. Write $\mathcal{P}_0$ for the constant functions and $\mathcal{P}_1$ for the affine functions $\mathbf{x} \mapsto a_0 + \sum_{i=1}^d a_i x_i$.*

(a) *For LapD, $u^* = \sum_{k=1}^m x_{2k-1} x_{2k}$, one has $\|u^*\|^2 = (9m^2 + 7m)/144$ and*

$$\min_{c \in \mathcal{P}_0} \frac{\|u^* - c\|}{\|u^*\|} = \sqrt{\frac{7}{7+9m}}, \quad \min_{a \in \mathcal{P}_1} \frac{\|u^* - a\|}{\|u^*\|} = \sqrt{\frac{1}{7+9m}}.$$

(b) *For PoiD, $u^* = d^{-1/2} \sum_{i=1}^d \cos(\pi x_i)$, one has $\|u^*\|^2 = \frac{1}{2}$ for every d and*

$$\min_{c \in \mathcal{P}_0} \frac{\|u^* - c\|}{\|u^*\|} = 1, \quad \min_{a \in \mathcal{P}_1} \frac{\|u^* - a\|}{\|u^*\|} = \sqrt{1 - \frac{96}{\pi^4}} \approx 0.1203,$$

independently of d .

All proofs are in Supplementary Section S7.

3 Methods: PINN loss, Deep Ritz energy, collocation strategies, optimisers and error measures

This section fixes what is common to the experiments of Sections 4–6. Further details are in Supplementary Section S2, and the settings of every experiment in Table S14.

3.1 Networks and the PINN loss

Every approximant u_θ is a fully connected network with tanh activations and a linear output layer, with the default initialisation of PyTorch [43], un-normalised inputs and single-precision arithmetic (Table 2).

Table 2: Network, interior budget N_r and the terms of the PINN loss (7) for each problem. “ 2×128 ” means two hidden layers of 128 units. Each constraint block is one mean squared term; the number of points of a block is given in brackets. The initial displacement and the initial velocity of the wave problems are imposed at the same points.

Problem	Hidden	Parameters	N_r	Residual r_θ	Constraint blocks
Heat	2×128	17 025	256	$\partial_t u_\theta - \partial_{xx} u_\theta$	$u(0, \cdot)$ [100]; $u(\cdot, 0)$, $u(\cdot, 1)$ [100 each]
Wave1	3×64	8577	1024	$\partial_{tt} u_\theta - 4 \partial_{xx} u_\theta$	$u(0, \cdot)$, $\partial_t u(0, \cdot)$ [100]; $u(\cdot, 0)$, $u(\cdot, 1)$ [100 each]
Wave2	2×50	2801	512	$\partial_{tt} u_\theta - \Delta u_\theta$	$u(0, \cdot)$, $\partial_t u(0, \cdot)$ [21 $\times$ 21]; u on the 4 side faces [11 $\times$ 21 each]
Lap2	3×128	33 537	256	Δu_θ	u at $x = 0$, $x = \pi$ [100 each]; $\partial_y u$ at $y = 0$, $y = \pi$ [100 each]
Lap3, Lap3s	2×100	10 601	512	Δu_θ	u on the 6 faces [12 $\times$ 12 each]
LapD, PoiD	3×64	$64d + 8449$	1024	$\Delta u_\theta + f$	u on $\partial\Omega$ [1024], weight λ

A PINN [47] is trained by minimising the mean squared residual of the differential equation at N_r interior points $\mathbf{x}_i$ plus one mean squared term for each of the K blocks of initial or boundary data,

$$\mathcal{L}(\theta) = \frac{1}{N_r} \sum_{i=1}^{N_r} r_\theta(\mathbf{x}_i)^2 + \lambda \sum_{k=1}^K \frac{1}{N_k} \sum_{j=1}^{N_k} \left(B_k u_\theta(\mathbf{y}_j^{(k)}) - g_k(\mathbf{y}_j^{(k)}) \right)^2. \quad (7)$$

Here a block is one edge, one face or one initial condition, with its points $\mathbf{y}_j^{(k)}$, its datum g_k and its operator B_k (the identity, ∂_t for an initial velocity, ∂_y for a Neumann edge); for the evolution problems the points are space–time points. Because every block is a mean, blocks with different numbers of points weigh the same. In Sections 4 and 5 we take $\lambda = 1$ and tune nothing; in Section 6 there is one boundary block and λ is chosen as described in Section 3.5. All derivatives in r_θ and $B_k u_\theta$ are computed by reverse-mode automatic differentiation of the network output with respect to the *full* input tensor, and the required component is selected by indexing the result. Differentiating with respect to a slice of the input is not equivalent: the slice is a new tensor on which the output does not depend (Section 7.1).

3.2 Collocation strategies

The interior points $\mathbf{x}_i$ are placed by one of the six strategies of Table 3, always with the same number N_r of points. The budgets $N_r = 256 = 16^2$ (Heat, Lap2), $1024 = 32^2$ (Wave1) and $512 = 8^3$ (Wave2, Lap3) are perfect powers, as a tensor grid requires, and powers of two, as suits a Sobol’ set. They are deliberately small, so that the placement of the points can matter; Section 4 also varies N_r for Heat. The constraint points are identical for every strategy: 100 equispaced points on each edge or initial line of the two-dimensional domains, a 21×21 initial grid and 11×21 points on each side face for Wave2, and an open 12×12 cell-centred grid on each face for Lap3, which keeps away from the two edges where its data are discontinuous.

For Heat the cell-centred grid has its first column of points at $t = 1/32$ and so leaves the strip $0 < t < 1/32$ next to the initial line without a residual point; 14 of the 256 random points and 8 of the 256 Sobol points of seed 0 lie in that strip. The node-centred grid has a column on

Table 3: Collocation strategies for the N_r interior points. “Coordinate” includes time for the evolution problems; all sets are mapped affinely to the domain. A seed fixes the sets of the four random strategies.

Strategy	Construction	During training
grid-c	cell-centred tensor grid: with D coordinates and $q = N_r^{1/D}$, the points $(j + \frac{1}{2})/q$, $j = 0, \dots, q - 1$, in each coordinate; no point on the boundary or on the initial line	fixed
grid-n	node-centred closed tensor grid: the points $j/(q - 1)$, $j = 0, \dots, q - 1$, in each coordinate; includes the boundary and the initial line. Used only in the controls of Section 4.3	fixed
random	independent uniform points, drawn once	fixed
resample	independent uniform points	redrawn before every Adam iteration; one further draw, then fixed, for L-BFGS
Sobol	first N_r points of a scrambled Sobol’ sequence [42, 54], generated with SciPy [57]	fixed
RAD	residual-based adaptive distribution of Wu et al. [63]: starting from the Sobol set, N_r points are drawn without replacement from a uniform pool of $8N_r$ points with probability proportional to $ r_\theta /\text{mean} r_\theta + 1$	redrawn every 500 Adam iterations and once before L-BFGS

the initial line itself but none inside the strip: its next column is at $t = 1/15$ (Figure S1). This matters in Section 4.3.

3.3 Optimiser arms

Two arms of 3000 iterations each are compared, an iteration being one update of the parameters. The arm Adam runs Adam [25] with the constant learning rate 10^{-3} for 3000 iterations. The arm Adam $\rightarrow$ L-BFGS shares the first 1500 of these iterations—it branches from a stored copy of the same run—and continues with 1500 iterations of L-BFGS [32] with memory 50 on a fixed point set, a common schedule [34]. For a single network, with the L-BFGS of PyTorch and a strong-Wolfe line search, L-BFGS needed between 1.045 and 1.065 loss-and-gradient evaluations per iteration on the five problems, so an equal number of iterations is close to an equal cost. The multi-seed runs use our own L-BFGS for a stack of networks (Supplementary Section S2.2); the comparison of the arms is therefore one at equal iterations.

3.4 The Deep Ritz method and its boundary penalty

For the Dirichlet problem $-\Delta u = f$ in $\Omega = (0, 1)^d$, $u = g$ on $\partial\Omega$, the Deep Ritz method [12] minimises over the network parameters the energy whose Euler–Lagrange equation is the differential equation, with the boundary condition imposed by a penalty of weight β :

$$\mathcal{E}_\beta(v) = \int_{\Omega} \left(\frac{1}{2} |\nabla v|^2 - f v \right) d\mathbf{x} + \beta \int_{\partial\Omega} (v - g)^2 ds. \quad (8)$$

With N_r points $\mathbf{x}_i$ uniform in Ω and N_b points $\mathbf{y}_j$ uniform on $\partial\Omega$, and because $|\Omega| = 1$ and $|\partial\Omega| = 2d$, the loss is the Monte-Carlo estimate

$$\frac{1}{N_r} \sum_{i=1}^{N_r} \left(\frac{1}{2} |\nabla u_\theta(\mathbf{x}_i)|^2 - f(\mathbf{x}_i) u_\theta(\mathbf{x}_i) \right) + \frac{2d\beta}{N_b} \sum_{j=1}^{N_b} (u_\theta(\mathbf{y}_j) - g(\mathbf{y}_j))^2. \quad (9)$$

Only first derivatives of the network enter. Unlike the PINN loss, the minimum of this loss is not zero and is not known in advance, so its raw value is not an error indicator: only its excess over the minimum measures an error, namely $\mathcal{E}_\beta(v) - \mathcal{E}_\beta(u_\beta) = \frac{1}{2} \|\nabla(v - u_\beta)\|_{L^2(\Omega)}^2 + \beta \|v - u_\beta\|_{L^2(\partial\Omega)}^2$, where u_β is the minimiser of $\mathcal{E}_\beta$. The distance measured is that to u_β , not to the solution of the Dirichlet problem.

The penalty changes the problem that is solved. This is classical [3] and has been analysed for the Deep Ritz method by Müller and Zeinhofer [39]. We claim no novelty for the following statement; its proof on the cube is elementary and is included to keep the article self-contained (Supplementary Section S7.4).

Proposition 2 (Penalised Ritz energy: Robin problem and penalty bias). *Let $\Omega = (0, 1)^d$, $f \in L^2(\Omega)$, $g \in L^2(\partial\Omega)$ and $\beta > 0$, and let $\mathcal{E}_\beta$ be the penalised Ritz energy (8), defined for $v \in H^1(\Omega)$.*

(a) *$\mathcal{E}_\beta$ has a unique minimiser $u_\beta \in H^1(\Omega)$. It is characterised by*

$$\int_{\Omega} \nabla u_\beta \cdot \nabla \varphi d\mathbf{x} + 2\beta \int_{\partial\Omega} (u_\beta - g) \varphi ds = \int_{\Omega} f \varphi d\mathbf{x} \quad \text{for all } \varphi \in H^1(\Omega),$$

which is the weak form of the Robin problem $-\Delta u = f$ in Ω , $\partial_n u + 2\beta(u - g) = 0$ on $\partial\Omega$.

(b) *Assume that the Dirichlet problem $-\Delta u = f$ in Ω , $u = g$ on $\partial\Omega$ has a solution $u^* \in C^2(\bar{\Omega})$, and put $w = u_\beta - u^*$. Then*

$$\|\nabla w\|_{L^2(\Omega)}^2 + 2\beta \|w\|_{L^2(\partial\Omega)}^2 = - \int_{\partial\Omega} (\partial_n u^*) w ds,$$

and consequently

$$\|w\|_{L^2(\partial\Omega)} \leq \frac{\|\partial_n u^*\|_{L^2(\partial\Omega)}}{2\beta}, \quad \|\nabla w\|_{L^2(\Omega)}^2 \leq \frac{\|\partial_n u^*\|_{L^2(\partial\Omega)}^2}{8\beta}.$$

(c) Under the assumption of (b),

$$\|u_\beta - u^*\|_{L^2(\Omega)} \leq \frac{C_d \|\partial_n u^*\|_{L^2(\partial\Omega)}}{\beta} \quad \text{for all } \beta > 0, \quad \text{with } C_d = \sqrt{1 + \frac{1}{d\pi^2}} \leq 1.05.$$

(d) Under the assumption of (b), $u_\beta = u^*$ for one (equivalently, for every) $\beta > 0$ if and only if $\partial_n u^* = 0$ on $\partial\Omega$.

For **LapD** one has $\|\partial_n u^*\|_{L^2(\partial\Omega)}^2 = 4m/3$ with $m = \lfloor d/2 \rfloor$, so the penalised minimiser is biased for every finite β ; for **PoiD** one has $\partial_n u^* = 0$, so $u_\beta = u^*$ for every $\beta > 0$.

On the cube the constant of part (c) is sharpened by Corollary 7(b) in Section 6.7. For **LapD** at $d = 2$, a finite-difference solution of the Robin problem shows the relative bias $\|u_\beta - u^*\|/\|u^*\|$ to decay like $0.50/\beta$ to $0.55/\beta$ for $\beta \geq 10$, that is, the bias $\|u_\beta - u^*\|$ itself like $0.167/\beta$ to $0.185/\beta$ (Observation S3). The bias is the error of the limit of the training, not a lower bound on the error of a trained network, which can end on either side of it.

Remark 3. In contrast, consider the population PINN loss, the limit of the loss (7) with one boundary block as the numbers of points grow. Because every term of (7) is a mean, and $|\Omega| = 1$ and $|\partial\Omega| = 2d$ for the unit cube, it is

$$\mathcal{L}_\lambda^{\text{POP}}(v) = \|\Delta v + f\|_{L^2(\Omega)}^2 + \frac{\lambda}{|\partial\Omega|} \|v - g\|_{L^2(\partial\Omega)}^2.$$

For a function $v \in C^2(\overline{\Omega})$ it vanishes if and only if v solves the Dirichlet problem, for every $\lambda > 0$: the exact solution is a global minimiser whatever the weight. The weight λ changes the conditioning of the optimisation problem but not its solution over $C^2(\overline{\Omega})$, or over any class of functions that contains u^* , whereas β changes the solution of the penalised Ritz problem itself. Over a class that does not contain u^* , such as a network of fixed size, the minimiser of $\mathcal{L}_\lambda^{\text{POP}}$ does in general depend on λ , because the residual and the boundary misfit are traded against each other. The two weights are also normalised differently: λ multiplies a mean over the boundary and β an integral over it, so that one boundary sample carries the weight λ/N_b in (7) and $2d\beta/N_b$ in (9). At fixed (λ, β) the boundary weight of Deep Ritz relative to that of the PINN therefore grows like $2d$.

3.5 Protocol in d dimensions

For **LapD** and **PoiD** the PINN and the Deep Ritz method use the same network, optimiser and points and differ only in the loss: (7) with residual $\Delta u_\theta + f$ and one boundary block of weight λ , or (9). At every iteration $N_r = 1024$ interior and $N_b = 1024$ boundary points are drawn afresh, as in [53]; a boundary point is obtained by choosing one of the $2d$ faces uniformly. The optimiser is Adam with learning rate 10^{-3} , multiplied by 0.1 after 50 and after 75 per cent of the run, for 4000 iterations unless stated otherwise. For a given seed the two methods start from the same network and see the same points. The weights were chosen once for each problem and method, from $\{1, 10, 100, 1000\}$, at $d = 5$, by the error of one run with a separate seed on a validation set of 5000 points: $\lambda = 1000$ for the PINN on both problems, $\beta = 100$ for Deep Ritz on **LapD** and $\beta = 1$ on **PoiD**. They are then used for every d . Because λ weights a boundary mean and β a boundary integral (Remark 3), keeping both fixed means that the coefficient of one boundary sample in the Deep Ritz loss is $2d\beta/\lambda$ times its coefficient in the PINN loss: 0.4 at $d = 2$ and 4 at $d = 20$ on **LapD**, and 0.004 and 0.04 on **PoiD**. The comparison across d in Section 6 is made under this convention; a convention that fixed the weight per unit of boundary measure for

both methods was not tried. The Laplacian in the PINN residual is computed by propagating $(u, \nabla u, \Delta u)$ forward through the layers [30]; nested reverse-mode differentiation gives the same loss at a higher cost (Supplementary Section S2.4).

3.6 Error measures and trivial predictors

Errors are measured on the held-out evaluation points $\mathbf{z}_1, \dots, \mathbf{z}_M$ of Table 1, and always for the network at the last iteration: no checkpoint is selected. With $\|v\|_2^2 = \sum_{i=1}^M |v(\mathbf{z}_i)|^2$ and $\bar{u}^* = M^{-1} \sum_i u^*(\mathbf{z}_i)$,

$$\begin{aligned} \varepsilon_2 &= \frac{\|u_\theta - u^*\|_2}{\|u^*\|_2}, & \varepsilon_\infty &= \max_{1 \leq i \leq M} |u_\theta(\mathbf{z}_i) - u^*(\mathbf{z}_i)|, & \varepsilon_2^c &= \frac{\|u_\theta - u^*\|_2}{\|u^* - \bar{u}^*\|_2}, \\ \varepsilon_{H^1} &= \frac{\|\nabla u_\theta - \nabla u^*\|_2}{\|\nabla u^*\|_2}, & \varepsilon_2^{\text{slice}}(t) &= \frac{\|u_\theta(t, \cdot) - u^*(t, \cdot)\|_{2,t}}{\|u^*(t, \cdot)\|_{2,t}}, \end{aligned} \tag{10}$$

where $\|\cdot\|_{2,t}$ sums over the evaluation points of the time slice t only. We report ε_2 and ε_∞ for every problem; the centred error ε_2^c and the H^1 -seminorm error ε_{H^1} in Section 6; and the slice error $\varepsilon_2^{\text{slice}}$ for Heat. A relative error has to be read against what a trivial predictor achieves: the zero function has $\varepsilon_2 = 1$, the best constant has $\varepsilon_2^c = 1$, and Proposition 1 gives the errors of the best constant and of the best affine function for LapD and PoiD; for LapD they decrease with d , so a small ε_2 in high dimension is not by itself evidence of an accurate solution.

3.7 Seeds and statistical procedure

A seed fixes the initial weights and every random point set of a run. We use 10 seeds for Heat and Lap2, 5 for Wave1, Wave2 and Lap3, and 3 in Section 6; other counts are stated where they occur. A *network* is one combination of problem, strategy and seed, trained up to the iteration at which the optimiser arms branch; a *run* is one network in one arm; a *pair* is the two runs of one network. The benchmark thus has 175 networks, 175 pairs and 350 runs. The strategies of one seed start from the same network and are compared *paired by seed*. We report means with sample standard deviations over seeds, and geometric means; the ratio of two strategies or arms is the geometric mean of the per-seed ratios.

The test is the exact two-sided Wilcoxon signed-rank test [62] on the per-seed differences of $\log \varepsilon_2$. With n seeds its smallest attainable p -value is 2^{1-n} , reached when all seeds agree in direction: $2/1024 \approx 0.002$ for 10 seeds and 0.0625 for 5. For the comparison of the optimiser arms the errors of the five strategies are first averaged geometrically within each seed, because they share an initial network. The p -values are not corrected for multiple comparisons. We call an effect *significant* only if it has the smallest attainable ten-seed value, $p = 2/1024$, that is, only if all ten seeds agree in direction. There are 24 signed-rank tests with ten seeds in the article (16 strategy comparisons, 2 optimiser comparisons and 6 tests of the control experiment of Section 4.3); by the Bonferroni inequality the probability that one or more of them is called significant when no effect exists is at most $24 \times 2/1024 = 4.7$ per cent. Nothing measured with fewer than ten seeds can be called significant by this rule; for those comparisons we report directions and ranges, and their p -values are descriptive. With 3 seeds (Section 6) no test is made, and statements are restricted to cases in which the seed ranges of the two methods are disjoint. The rule was fixed after the results were known; it is stated so that it can be judged, not because it was pre-registered. The looser threshold $p \leq 0.004$ would also admit effects on which nine of ten seeds agree and the one dissenting seed has the smallest difference (signed-rank statistic $W = 1$, $p = 4/1024$); two such comparisons exist, Sobol' points on Lap2 and the repeated comparison of the cell-centred grid with random points in the control experiment of Section 4.3, and we call them suggestive, not significant.

3.8 Finite-difference baselines

Each low-dimensional problem is also solved, on a sequence of grids, by a standard finite-difference scheme of second order [29]: the Crank–Nicolson scheme [9] with $\Delta t = \Delta x$ for **Heat**; the explicit leapfrog scheme with Courant number 0.5 and a second-order starting step for **Wave1** and **Wave2** (for **Wave1** the Courant number 1 is avoided because the scheme is then exact at the nodes, which would not be a representative baseline); the five-point Laplacian with second-order ghost-point Neumann rows and a sparse direct solver for **Lap2**; and the seven-point Laplacian, solved with a discrete sine transform, for **Lap3**. The grid solution is evaluated on the same held-out points as the networks, by its nodal values where these points are nodes and by cubic spline interpolation otherwise.

3.9 Verification codes and pre-registration

Two bodies of code, written separately from the code of the experiments, are used to check the results, and are referred to by name. **V-bench** re-implements the low-dimensional PINNs: one network at a time on the CPU, derivatives propagated analytically through the layers, the strong-Wolfe L-BFGS of PyTorch, seeds 0 to 5. It re-ran 40 configurations of Sections 4 and 5. **V-dim** re-implements the d -dimensional study: its own exact solutions, a nested Laplacian, its own test points, seeds 7 to 9 (7 to 11 in one cell); it trained 73 networks for Section 6. Both also re-aggregated the stored raw results (Supplementary Section S8.5).

The verification has two limits. First, the codes were written, and the stored results re-aggregated, separately from the experiments but within the same project, with the aim of refuting the results; they ran on the same machine, with the same library versions, as the experiments. This guards against errors of one code and of one analysis; it is not a replication by another group on other hardware, and errors common to both sides (a shared misreading of a problem, the single-precision library stack) are not excluded. Second, the following experiments were run with the code of the experiments only and are not covered: the longer runs on **Wave2** and **Wave1** (Sections 5.1 and 5.3), the compatible-data problem **Lap3s** (Section 5.2), the slice errors of **Heat** (Section 4.5), the control experiment for the cell-centred grid (Section 4.3), the 16 000-iteration runs at $d = 20$ (Section 6.3) and the demonstration of Section 7.1. The demonstration of Section 7.2 was re-run for three of its five seeds with the network and derivative code of **V-bench**. The experiments listed above have only internal cross-checks: the configurations that they share with the benchmark agree with it, **Lap3s** is run with its own control, and the drivers are reproducible bit for bit on the machine used (Supplementary Section S8.4).

The analyses of Sections 6.7 and 6.8 were pre-registered: hypotheses, networks or problems, seeds and decision rules were fixed in a document whose hash and time were recorded in the repository (`PREREG_FREEZE.json`) before the first counted evaluation or run; no external registry was used. Each was then re-checked, in two rounds, with separately written code within the same project and on the same machine: the proofs were re-read and their formulas checked numerically, every quoted number was recomputed from the stored files, networks were retrained with other seeds, and problems were added on which neither remedy had been tried. Below, the two rounds are referred to together as *the re-check*; it has the limits stated above; where it added a result, the text says so, and Supplementary Section S8.5 describes it.

4 Low-dimensional benchmark: accuracy, optimiser and collocation strategy

We now apply the protocol of Section 3 to the five low-dimensional problems. Every seed is combined with each of the five strategies `grid-c`, `random`, `resample`, `Sobol` and `RAD`: 10×5 networks for **Heat** and for **Lap2**, and 5×5 for each of **Wave1**, **Wave2** and **Lap3**, 175 in all. Each network is trained in both optimiser arms, which gives 175 pairs and 350 runs. This section treats

what the optimiser and the collocation strategy change, on all five problems; what the errors on Wave1, Wave2 and Lap3 mean, and the comparison with finite differences, are the subject of Section 5. Several statements below are accompanied by the result of the verification code V-bench (Section 3.9). On the 27 configurations that it shares with the benchmark, the ratio of its Adam $\rightarrow$ L-BFGS error to the benchmark’s has geometric mean 0.98 and range 0.60 to 1.26.

4.1 Accuracy

After Adam $\rightarrow$ L-BFGS the heat problem is solved to a mean ε_2 between 1.16×10^{-3} and 1.45×10^{-3} by the four strategies other than grid-c, and to $6.16 \times 10^{-3} \pm 4.01 \times 10^{-3}$ by grid-c; the Laplace problem Lap2 is solved to between 2.41×10^{-4} and 4.87×10^{-4} by all five (Table 4). Wave2 and Lap3 end with strategy means between 3.6 and 5.0 per cent (single runs 2.7 to 8.2 per cent) and Wave1 above 40 per cent; these three are discussed in Section 5.

4.2 Optimiser

Replacing the second half of the Adam iterations by L-BFGS lowers the error on every problem (Figure S3). In geometric mean over strategies and seeds, ε_2 falls from 2.12×10^{-2} to 1.56×10^{-3} on Heat (ratio 0.074), from 7.78×10^{-3} to 3.39×10^{-4} on Lap2 (0.044), from 0.208 to 0.0473 on Wave2 (0.228), from 0.145 to 0.0420 on Lap3 (0.291), and from 0.483 to 0.442 on Wave1 (0.915), where neither arm succeeds. The Adam $\rightarrow$ L-BFGS arm is the better one in all 175 pairs. Tested per seed, it is better in 10 of 10 seeds on Heat and on Lap2 ($p = 0.002$, significant) and in 5 of 5 seeds on each of the other three ($p = 0.0625$, the smallest value attainable with five seeds). In V-bench it is the better arm in 39 of 40 pairs; the exception is the closest pair of the benchmark on Heat (Supplementary Section S3.1).

The Adam arm is the last iterate of a run at constant learning rate, and that iterate is noisy: between iterations 1500 and 3000 the largest and the smallest logged held-out error of one Adam run differ by a median factor of 8.0 on Heat and of 28 on Lap2. The conclusion does not rest on that end point. On the subset of the evaluation points on which the training curves are logged, the final Adam $\rightarrow$ L-BFGS error is 0.074 (Heat) and 0.044 (Lap2) times the last Adam error, 0.076 and 0.034 times the median of the logged Adam errors of the same run, and 0.124 and 0.077 times the smallest of them, in geometric mean. The smallest logged error is an oracle checkpoint, selected with knowledge of the exact solution; the Adam $\rightarrow$ L-BFGS arm is below even that in 49 of 50 runs on Heat and in 50 of 50 on Lap2. The gain is therefore about one order of magnitude: a factor 14 to 23 against the last Adam iterate and 8 to 13 against the best logged one. The Adam arm has no learning-rate schedule; one decaying schedule, tried in V-bench, did

Table 4: Held-out relative L^2 error ε_2 of the benchmark: mean $\pm$ sample standard deviation over seeds, in units of the power of ten in the column “unit”. Both arms have 3000 iterations; Adam $\rightarrow$ L-BFGS is 1500 Adam iterations followed by 1500 L-BFGS iterations. Unit loss weights. The L^∞ errors and the geometric means are in Table S15; every run is shown in Figure S2.

	N_r	seeds	arm	unit	grid-c	random	resample	Sobol	RAD
Heat	256	10	Adam	10^{-2}	2.98 ± 2.98	4.33 ± 6.43	3.70 ± 1.90	2.41 ± 2.00	1.48 ± 0.52
			Adam $\rightarrow$ L-BFGS	10^{-3}	6.16 ± 4.01	1.25 ± 0.39	1.45 ± 1.00	1.22 ± 0.41	1.16 ± 0.52
Lap2	256	10	Adam	10^{-3}	6.00 ± 4.56	8.66 ± 9.88	50.89 ± 33.81	4.08 ± 1.62	11.22 ± 14.70
			Adam $\rightarrow$ L-BFGS	10^{-4}	2.41 ± 0.83	4.87 ± 1.65	4.40 ± 1.58	2.55 ± 0.72	4.35 ± 1.62
Wave1	1024	5	Adam	10^{-1}	4.83 ± 0.21	4.81 ± 0.22	4.86 ± 0.22	4.82 ± 0.22	4.84 ± 0.22
			Adam $\rightarrow$ L-BFGS	10^{-1}	4.35 ± 0.15	4.42 ± 0.14	4.41 ± 0.19	4.43 ± 0.11	4.49 ± 0.12
Wave2	512	5	Adam	10^{-1}	2.11 ± 0.68	2.13 ± 0.65	2.35 ± 0.65	2.09 ± 0.57	2.03 ± 0.44
			Adam $\rightarrow$ L-BFGS	10^{-2}	4.92 ± 1.26	4.89 ± 1.92	5.01 ± 0.87	4.85 ± 1.72	4.79 ± 1.32
Lap3	512	5	Adam	10^{-1}	1.18 ± 0.22	3.00 ± 3.43	1.55 ± 0.20	1.31 ± 0.15	1.33 ± 0.18
			Adam $\rightarrow$ L-BFGS	10^{-2}	4.88 ± 1.21	4.63 ± 0.88	4.28 ± 0.37	3.56 ± 0.37	4.03 ± 0.62

not close the gap (Supplementary Section S3.1), but it is not a tuned Adam baseline, and we did not search for one. That an L-BFGS phase after Adam helps a PINN is known [34], and Rathore et al. [48] measured by how much: with five seeds, a tuned Adam rate and 41 000 iterations on convection, reaction and wave problems, their Adam $\rightarrow$ L-BFGS run with the lowest loss has an error 14 times below that of their Adam run with the lowest loss on the convection problem. What is added here is the same measurement, paired by seed, on the heat and Laplace problems at a small budget and against an untuned Adam.

4.3 Collocation strategy

Table 5 compares each strategy with **random**, paired by seed. No strategy is best throughout. Three entries of the benchmark block are significant in the sense of Section 3.7 (all ten seeds agree): the cell-centred grid on **Heat** (worse), the cell-centred grid on **Lap2** (better) and resampling under Adam on **Lap2** (worse). A fourth, Sobol’ points on **Lap2**, is suggestive (nine of ten seeds). Each needs a qualification; the first does not keep its label when the comparison is repeated, as described next.

The cell-centred grid on the heat problem. After Adam $\rightarrow$ L-BFGS, **grid-c** is 4.21 times worse than **random** on **Heat**, in 10 of 10 seeds ($p = 0.002$). The cell-centred grid has no residual point in the strip $0 < t < 1/32$ (Section 3.2), where the solution decays fastest, and its error sits there: in seed 0, 70 per cent of its squared error on the evaluation grid lies in $t \leq 0.05$, against 9, 17 and 5 per cent for **random**, **Sobol** and **RAD** (Figure S4).

Whether the empty strip is the cause was tested directly (Table 6). The heat runs were repeated with the driver of the benchmark and seeds 0 to 9 for six point sets that differ in one respect at a time: **random**; **grid-c**; the node-centred grid **grid-n**; **grid-c** shifted by half a cell in t only, which puts its first column on the initial line; **grid-c** with one further column of 16 points at $t = 1/64$, inside the strip; and, as a placebo, **grid-c** with one further column of 16 points at $t = 1/2$. The two sets shared with the benchmark reproduce it (the errors are 0.94 and 0.97 times those of the benchmark runs of the same seeds, in geometric mean): **grid-c** is 4.31 times worse than **random**. One seed reverses (seed 3: ratio 0.94, against 1.09 in the benchmark), so that the comparison gives $p = 0.004$ and does not meet our rule. The two experiments differ in the

Table 5: Collocation strategies compared with **random** at the same N_r and the same seed: geometric mean over seeds of the ratio of ε_2 (a value below 1 means better than **random**), followed in brackets by the number of seeds in which the strategy is the better one and, where a test was made, the exact two-sided Wilcoxon p -value, which is not corrected for multiple comparisons (Section 3.7). The lower block is the verification code **V-bench**; there the row **grid-n** replaces the cell-centred grid by the node-centred grid. A dash means not run.

	arm	grid-c	resample	Sobol	RAD
<i>Benchmark (10 seeds for Heat, Lap2; 5 seeds for Wave1, Wave2, Lap3)</i>					
Heat	Adam $\rightarrow$ L-BFGS	4.21 (0/10; 0.002)	1.05 (6/10; 1.000)	0.97 (6/10; 0.695)	0.90 (5/10; 0.695)
Lap2	Adam $\rightarrow$ L-BFGS	0.49 (10/10; 0.002)	0.91 (6/10; 0.322)	0.54 (9/10; 0.004)	0.88 (8/10; 0.160)
Wave1	Adam $\rightarrow$ L-BFGS	0.98 (4/5; 0.438)	1.00 (3/5; 0.812)	1.00 (3/5; 1.000)	1.02 (1/5; 0.188)
Wave2	Adam $\rightarrow$ L-BFGS	1.04 (3/5; 1.000)	1.07 (2/5; 0.812)	0.99 (2/5; 1.000)	1.00 (3/5; 1.000)
Lap3	Adam $\rightarrow$ L-BFGS	1.04 (2/5; 0.812)	0.93 (3/5; 0.438)	0.78 (5/5; 0.0625)	0.88 (3/5; 0.625)
Heat	Adam	0.86 (5/10; 0.846)	1.33 (4/10; 0.275)	0.71 (6/10; 0.432)	0.57 (5/10; 0.432)
Lap2	Adam	0.79 (6/10; 0.322)	6.36 (0/10; 0.002)	0.61 (7/10; 0.084)	1.00 (5/10; 1.000)
<i>Verification code V-bench (6 or 3 seeds)</i>					
Heat	Adam $\rightarrow$ L-BFGS	3.94 (0/6; 0.031)	—	0.90 (2/3)	—
Heat, grid-n	Adam $\rightarrow$ L-BFGS	1.26 (2/6; 0.438)	—	—	—
Lap2	Adam $\rightarrow$ L-BFGS	0.75 (2/3)	1.56 (1/3)	0.84 (2/3)	—
Lap2	Adam	—	2.73 (0/3)	—	—

Table 6: Control experiment on problem Heat: Adam $\rightarrow$ L-BFGS, seeds 0 to 9, the driver and protocol of the benchmark. N_r : number of interior points. “Shifted to $t = 0$ ”: the cell-centred grid moved by half a cell in t only, so that its columns are at $t = j/16$, $j = 0, \dots, 15$, and its rows stay at $x = (j + \frac{1}{2})/16$. “+ column, $t = \dots$ ”: the cell-centred grid with 16 further points at the stated t and the x of the grid. Ratios are geometric means of per-seed ratios of ε_2 ; in brackets the number of seeds in which the set is the better one and, against random, the exact two-sided Wilcoxon p -value. Last column: share of the squared error on the evaluation grid that lies in $t \leq 0.05$, median over seeds (range), in per cent.

point set	N_r	ε_2		ratio of ε_2 to		error in $t \leq 0.05$ (%)
		geo. mean	mean $\pm$ s.d.	random	grid-c	
random	256	1.12×10^{-3}	$(1.19 \pm 0.43) \times 10^{-3}$	—	—	25 (7–43)
grid-c	256	4.84×10^{-3}	$(6.07 \pm 4.08) \times 10^{-3}$	4.31 (1/10; 0.004)	—	71 (29–73)
grid-n	256	2.22×10^{-3}	$(2.76 \pm 1.95) \times 10^{-3}$	1.98 (2/10; 0.049)	0.46 (6/10)	23 (8–35)
grid-c shifted to $t = 0$	256	2.07×10^{-3}	$(2.37 \pm 1.24) \times 10^{-3}$	1.84 (3/10; 0.064)	0.43 (7/10)	31 (6–40)
grid-c + column, $t = 1/64$	272	1.04×10^{-3}	$(1.08 \pm 0.36) \times 10^{-3}$	0.92 (5/10; 0.625)	0.21 (9/10)	11 (6–22)
grid-c + column, $t = 1/2$	272	5.36×10^{-3}	$(6.29 \pm 3.83) \times 10^{-3}$	4.77 (0/10; 0.002)	1.11 (3/10)	67 (41–72)

size and composition of the stack of networks trained together, which in floating point changes the trajectories of the L-BFGS phase (the mechanism was not investigated; Supplementary Section S8.4); a label that depends on such a detail is not robust, and we rest the claim on the placebo comparison. Sixteen points in the strip remove the penalty: the error is 0.92 times that of random (95 per cent interval 0.72 to 1.19, from a t -distribution on the logarithms of the ten ratios) and 0.21 times that of grid-c. The same sixteen points at $t = 1/2$ change nothing: 4.77 times random, worse in 10 of 10 seeds, and 1.11 times grid-c. The two augmented sets have the same 272 points except for the position of the extra column; the column in the strip gives the lower error in 10 of 10 seeds (ratio 0.19, interval 0.12 to 0.32, $p = 0.002$), which is significant. The penalty of the cell-centred grid is thus caused by the absence of residual points next to the initial line, not by the regularity of the grid. This agrees with the hypothesis of Daw et al. [11] that a PINN is trained by propagating the solution from the initial and boundary points into the interior, and with the case for causal training [61]; the control isolates the effect for one problem, one network and one budget. A column on the initial line itself is not enough. The node-centred grid grid-n is 1.98 times worse than random (worse in 8 of 10 seeds, $p = 0.049$) and the shifted grid 1.84 times (worse in 7 of 10, $p = 0.064$); each has about 0.45 times the error of grid-c, and neither difference from random is significant by our rule (V-bench, 6 seeds: grid-n 1.26 times random). A grid with a column on the initial line thus loses about half of the penalty in geometric mean, with a large spread between seeds; both grids have their second column at $t = 1/16$ or $1/15$, by when the solution has lost nearly half of its amplitude ($e^{-\pi^2/15} = 0.52$).

The cell-centred grid and Sobol’ points on Lap2. On Lap2 the sign is reversed: grid-c has 0.49 times the error of random (10 of 10 seeds, $p = 0.002$) and Sobol 0.54 times (9 of 10, $p = 0.004$, suggestive). V-bench, with 3 seeds, finds the same direction but smaller effects, 0.75 and 0.84, each in 2 of 3 seeds. We have no tested explanation for the advantage of the two evenly spread sets on this problem.

Sobol’ points, RAD and resampling. The ratio of Sobol to random after Adam $\rightarrow$ L-BFGS does not exceed 1.00, to two decimals, on any of the five problems, but it is significant on none, suggestive only on Lap2, and it does not hold seed by seed (Sobol is the worse set in 4 of 10 seeds on Heat). RAD has ratios between 0.88 and 1.02 with all $p \geq 0.16$: no gain is detectable on any of the five problems, including Lap3, whose solution is not smooth. At our budget, however, the adaptive set is redrawn only four times and is fixed during the L-BFGS phase, in which most of the error is removed (Supplementary Section S3.2); the absence of a gain is a statement about our budget, not about residual-adaptive sampling. Under Adam alone, drawing fresh

points at every iteration is 6.36 times worse than a fixed random set on **Lap2** (worse in 10 of 10 seeds, $p = 0.002$; **V-bench** 2.73 times, worse in 3 of 3) and 1.33 times on **Heat**, which is not significant ($p = 0.28$). After the L-BFGS phase on a fixed set the benchmark shows no effect (ratios 0.91 to 1.07 on the five problems, all $p \geq 0.32$); the three **V-bench** seeds on **Lap2** give 1.56, so this neutrality is established only up to the spread of a few seeds. The penalty under Adam is measured without any decay of the learning rate.

On **Wave1**, **Wave2** and **Lap3** no difference between strategies can be significant, because they have five seeds. Wu et al. [63] found low-discrepancy points generally better than random points, both better than an equispaced grid, and periodically resampled random points better than fixed ones. Our **Sobol** result on **Lap2** is in the direction of the first statement; the grid ranks differently on **Lap2**, and we did not reproduce the benefit of resampling. Our setting (256 points, a cell-centred grid, redrawing at every iteration, a constant learning rate, 3000 iterations and four redraws of the adaptive set) differs from theirs, so the comparison is not like for like.

4.4 Point budget

How much the strategy matters depends on how many points there are. For **Heat** and the Adam $\rightarrow$ L-BFGS arm the runs of **grid-c**, **random** and **Sobol** were repeated with $N_r = 64$ and $N_r = 1024$, for seeds 0 to 4 (Table S1, Figure S5). With 64 points the spread between seeds is as large as the differences between strategies; with 256 points the penalty of the cell-centred grid is present in these five seeds too; and with 1024 points the three strategies agree, including the cell-centred grid, whose first column is then at $t = 1/64$. From 256 to 1024 points the errors of **random** and **Sobol** change by less than their spread between seeds, which suggests that on this problem, with this network and 3000 iterations, the error at 1024 points is limited by the optimisation and not by the collocation set. The sweep covers one problem and one arm.

4.5 Error on time slices of the heat problem

The space–time error ε_2 of **Heat** is normalised by a norm that is dominated by early times: 99 per cent of $\|u^*\|_2^2$ on the evaluation grid lies in $t \leq 0.25$. It therefore measures the error relative to the early-time amplitude. To see what a value of 10^{-3} means at later times we re-trained the **random** and **Sobol** configurations of **Heat** for seeds 0 to 9 with the single-network driver, kept the networks, and evaluated the slice error $\varepsilon_2^{\text{slice}}(t)$ of (10) on the 201 points of each of the 201 time levels of the evaluation grid, in double precision. The space–time errors of these runs agree with the benchmark (geometric means 1.07×10^{-3} and 1.16×10^{-3} after Adam $\rightarrow$ L-BFGS, against 1.19×10^{-3} and 1.15×10^{-3} for the runs of Table 4).

After Adam $\rightarrow$ L-BFGS the slice error is below 0.1 per cent up to $t = 0.1$, passes 1 per cent near $t = 0.4$ and 10 per cent near $t = 0.65$ to 0.68 (geometric means), and on the final slice it is 479 per cent for **random** (range over seeds 156 to 1241 per cent) and 567 per cent for **Sobol** (198 to 1298 per cent) (Table S2). In all 40 runs (20 networks, each in both arms) the error on the slice $t = 1$ exceeds 100 per cent, although the space–time error is about 0.1 per cent after Adam $\rightarrow$ L-BFGS and 1.6 to 2.1 per cent after Adam alone (Figure 1). The reason is visible in the absolute error: the root-mean-square error of a slice stays between 7.6×10^{-5} and 3.7×10^{-4} at every time level (geometric means over seeds, Adam $\rightarrow$ L-BFGS, both strategies), while the root-mean-square value of the solution on the slice falls from 0.71 to 3.6×10^{-5} . The network has a roughly uniform absolute accuracy, so $\varepsilon_2^{\text{slice}}(t)$ grows like $e^{\pi^2 t}$, and once the solution is smaller than that accuracy it is not resolved. A loss made of absolute residuals does not ask for more. Rescaling the unknown or weighting the loss in time might; neither was tried here. For a decaying solution a space–time relative error should therefore be reported together with the error per time slice.

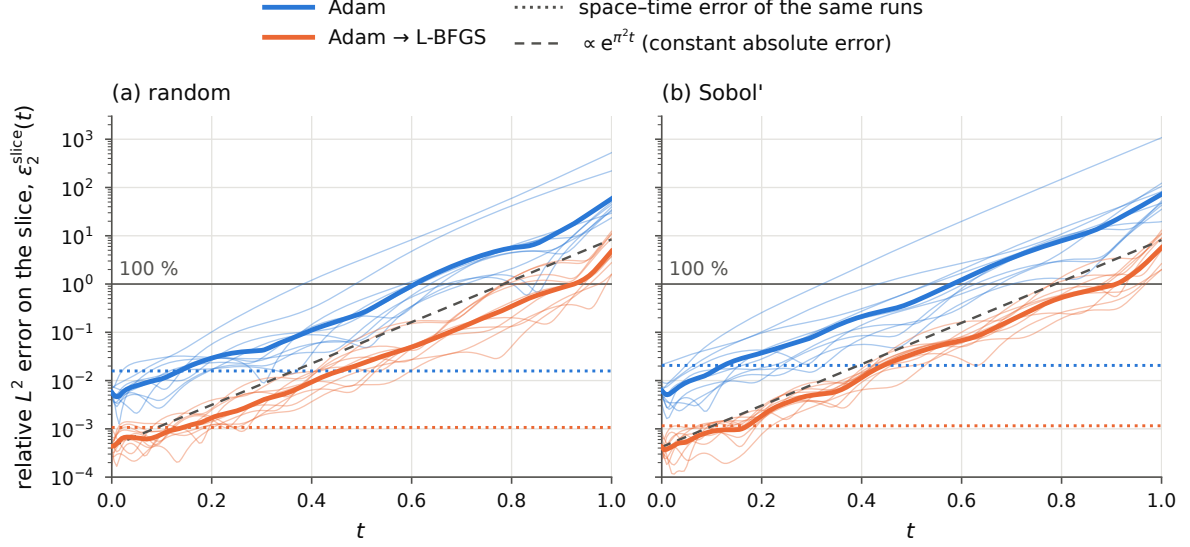


Figure 1: Relative L^2 error $\varepsilon_2^{\text{slice}}(t)$ on the time slice t of problem **Heat**, evaluated on 201 points in x at each of 201 time levels, for (a) *random* and (b) *Sobol'* points, $N_r = 256$, seeds 0 to 9, and the arms Adam (blue) and Adam $\rightarrow$ L-BFGS (orange). Thin lines: single seeds; thick lines: geometric mean over seeds. Dotted lines: geometric mean of the space-time error ε_2 of the same runs, in the colour of the arm. Dashed line: growth proportional to $e^{\pi^2 t}$, the slice error of a constant absolute error, drawn through the Adam $\rightarrow$ L-BFGS mean at $t = 0$. Horizontal line: 100 per cent.

5 Wave equations, the three-dimensional Laplace problem and classical baselines

Three of the five low-dimensional problems are not brought to the 10^{-3} level of Section 4 by the common protocol (unit weights, 1500 Adam and 1500 L-BFGS iterations): the two-dimensional wave problem **Wave2**, the three-dimensional Laplace problem **Lap3** and the two-mode wave problem **Wave1**. They fall short for three different reasons, which this section separates with three further experiments; it then sets all five problems against second-order finite differences. Table 7 collects the numbers.

5.1 The two-dimensional wave problem

With the benchmark protocol (2×50 network, $N_r = 512$, five seeds) the five strategy means of ε_2 on **Wave2** after Adam $\rightarrow$ L-BFGS lie between 4.79×10^{-2} and 5.01×10^{-2} , single runs between 2.7×10^{-2} and 8.2×10^{-2} , and Adam alone gives strategy means of 0.20 to 0.24. No strategy differs from *random* (paired ratios 0.99 to 1.07, all $p \geq 0.8$). These are not converged values: the median held-out error of the 25 runs is 7.24×10^{-2} after 1000 L-BFGS iterations and 4.58×10^{-2} after 1500, and it is still falling when the budget ends.

We therefore repeated the *Sobol'* configuration for seeds 0, 1 and 2 with a budget of 10 000 iterations: 1500 Adam iterations followed by up to 8500 L-BFGS iterations, and 10 000 Adam iterations as a comparator (single-network driver, single precision). At iteration 3000 these runs agree with the benchmark runs of the same seeds (Supplementary Section S4.1). In all three seeds the L-BFGS phase ended before the budget, on the optimiser's own termination test, after 8024, 8437 and 8345 iterations, with $\varepsilon_2 = 9.31 \times 10^{-3}$, 8.98×10^{-3} and 5.68×10^{-3} (mean 7.99×10^{-3} , standard deviation 2.00×10^{-3}) and ε_∞ between 1.7×10^{-2} and 2.4×10^{-2} : five to eight times below the error at iteration 3000. The curves have flattened but are not flat: over the last 1000 logged iterations before iteration 9500 the error still falls by 8 to 15 per cent. The 0.6 to 0.9 per cent quoted here is where this optimiser stopped in single precision, not a limit of the method;

Table 7: Held-out errors on the three harder problems. Benchmark rows (25 runs: five strategies, five seeds each): range of the five strategy means, with [min, max] over the 25 single runs underneath. Other rows: one strategy (Sobol), mean $\pm$ sample standard deviation over the seeds, with [min, max] over the seeds underneath. Every row has 3000 iterations per arm except the two “long runs”: 1500 Adam iterations followed by L-BFGS until the optimiser stopped (after 8024 to 8437 L-BFGS iterations for Wave2 and 7370 to 9224 for Wave1), and 10 000 (Wave2) or 41 000 (Wave1) Adam iterations for the Adam column. “Control”: Lap3 re-run with the driver and device used for Lap3s. Evaluation sets as in Table 1.

Problem, set-up	ε_2 , Adam	ε_2 , Adam $\rightarrow$ L-BFGS	ε_∞ , Adam $\rightarrow$ L-BFGS
Wave2, benchmark 25 runs	0.203–0.235 [0.148, 0.343]	4.79×10^{-2} – 5.01×10^{-2} [2.71×10^{-2} , 8.15×10^{-2}]	0.078–0.081 [0.050, 0.121]
Wave2, long run 3 seeds	$(8.80 \pm 2.55) \times 10^{-2}$ [0.065, 0.115]	$(7.99 \pm 2.00) \times 10^{-3}$ [5.68×10^{-3} , 9.31×10^{-3}]	$(1.98 \pm 0.33) \times 10^{-2}$ [1.74×10^{-2} , 2.36×10^{-2}]
Lap3, benchmark 25 runs	0.118–0.300 [0.083, 0.908]	3.56×10^{-2} – 4.88×10^{-2} [3.02×10^{-2} , 6.32×10^{-2}]	0.070–0.101 [0.059, 0.231]
Lap3, control 5 seeds	0.135 ± 0.037 [0.104, 0.193]	$(3.72 \pm 0.26) \times 10^{-2}$ [3.36×10^{-2} , 4.04×10^{-2}]	$(7.22 \pm 1.34) \times 10^{-2}$ [0.055, 0.091]
Lap3s 5 seeds	$(3.25 \pm 0.56) \times 10^{-2}$ [2.77×10^{-2} , 4.18×10^{-2}]	$(3.32 \pm 0.76) \times 10^{-3}$ [2.71×10^{-3} , 4.61×10^{-3}]	$(1.14 \pm 0.47) \times 10^{-2}$ [6.90×10^{-3} , 1.81×10^{-2}]
Wave1, benchmark 25 runs	0.481–0.486 [0.461, 0.512]	0.435–0.449 [0.414, 0.464]	0.652–0.690 [0.575, 0.745]
Wave1, long run 3 seeds	0.365 ± 0.020 [0.343, 0.380]	0.365 ± 0.024 [0.337, 0.381]	0.546 ± 0.012 [0.533, 0.557]

double precision, a restarted L-BFGS or more collocation points might lower it, and none was tested. Adam alone is at 6.5×10^{-2} to 0.115 after 10 000 iterations and still descending. The learnt function is a standing wave: at $(x, y) = (0.5, 0.5)$ it changes sign between $t = 0.35$ and $t = 0.40$ (exact zero at $t = 1/(2\sqrt{2}) \approx 0.354$) and ends at -0.264 , -0.263 and -0.266 at $t = 1$ (exact -0.266); the largest deviation over the 21 times and three seeds is 3.2×10^{-3} (Figure S6).

5.2 The three-dimensional Laplace problem: dimension or data?

For Lap3 (2×100 network, $N_r = 512$, 12×12 points per face) the strategy means after Adam $\rightarrow$ L-BFGS are 3.56×10^{-2} to 4.88×10^{-2} (single runs 3.0×10^{-2} to 6.3×10^{-2}), with ε_∞ between 0.07 and 0.10. The Dirichlet data of Lap3 are discontinuous along two edges (Proposition S1(b)), so no function that is continuous on the closed cube is uniformly accurate near them. A classical solver is affected too: the seven-point finite-difference scheme with $n = 20$ intervals per side (21^3 nodes) has the same L^2 accuracy as the PINN ($\varepsilon_2 = 3.90 \times 10^{-2}$, with $\varepsilon_\infty = 0.171$), and $n = 40$ (41^3 nodes) gives 9.97×10^{-3} and 0.0434. With the compatible datum of Lap3s the same scheme is 12 to 19 times more accurate on every grid tried.

To separate the third dimension from the discontinuity we solved Lap3s with the network, Sobol points, face points, optimiser settings, seeds (0 to 4) and evaluation grid of the benchmark, and re-ran Lap3 alongside as a control. The control reproduces the benchmark ($3.72 \times 10^{-2} \pm 2.6 \times 10^{-3}$ against $3.56 \times 10^{-2} \pm 3.7 \times 10^{-3}$). With compatible data the same set-up reaches $\varepsilon_2 = 3.32 \times 10^{-3} \pm 7.6 \times 10^{-4}$ (range 2.71×10^{-3} to 4.61×10^{-3}) and $\varepsilon_\infty = 1.14 \times 10^{-2} \pm 4.7 \times 10^{-3}$: seed by seed, the error on Lap3 is 8.8 to 14.1 times larger (geometric mean 11.4, all five seeds). In the Lap3 runs the 5 per cent of evaluation points closest to the two singular edges carry 67 to 79 per cent of the squared error (19 to 35 per cent for Lap3s). The several-per-cent error on Lap3 is therefore mainly a property of the data, not of the dimension (the data of Lap3s are compatible and also make the solution a single separable mode, so the comparison does not separate these two changes): with $N_r = 512$ points and 3000 iterations a three-dimensional harmonic function with compatible data is recovered to 0.3 per cent. That figure is not converged either (the median error falls from 4.1×10^{-3} to 3.1×10^{-3} over the last 250 iterations), and the conclusion rests on

one strategy and five seeds. The PINN’s factor between **Lap3** and **Lap3s** (geometric mean 11.4) is close to that of the finite-difference scheme, 12 on the grids on which the evaluation points are nodes.

5.3 The two-mode wave problem

Problem **Wave1** is not solved by the benchmark protocol. All 50 runs (3×64 network, $N_r = 1024$; 25 networks, two arms) end between 0.414 and 0.512 (Adam 0.461 to 0.512, Adam $\rightarrow$ L-BFGS 0.414 to 0.464). No strategy moves the error by more than 2 per cent (paired ratios to **random** 0.98 to 1.02). A predictor that reproduces the slow mode $\sin \pi x \cos 2\pi t$ and omits the fast one has $\varepsilon_2 = 1/\sqrt{5} \approx 0.447$ (the two modes are orthogonal with squared norms $1/4$ and $1/16$), and this is close to what the networks do: projected onto the two spatial modes, the stored predictions of seed 0 reproduce the slow mode to within 4 to 12 per cent, and the fast mode carries 84 to 93 per cent of the squared error (Supplementary Section [S4.2](#)).

Two published results bracket this outcome, and neither is a like-for-like comparison. Wang et al. [60], who introduced the problem, report 4.518×10^{-1} for the unweighted PINN trained with a first-order optimiser (five layers of 500 units, 80 000 iterations) and 1.728×10^{-3} with weights derived from the neural tangent kernel. Rathore et al. [48] train the unweighted loss on the same equation with a faster second mode ($\sin 5\pi x$ in place of $\sin 4\pi x$), three hidden layers of 50 to 400 units, 10 000 residual points and 41 000 iterations: the run with the lowest loss after tuning reaches 0.349 with Adam, 0.335 with L-BFGS, 0.055 with Adam followed by L-BFGS (their Table 1) and 0.013 after a further Newton–conjugate-gradient phase (their Table 2). The unweighted loss can therefore be brought to a few per cent, and the level 0.41 to 0.51 of our benchmark is not a limit of the formulation.

Whether the budget alone is responsible was tested by repeating the Sobol configuration for seeds 0, 1 and 2 with their budget of 41 000 iterations (single-network driver, single precision): 41 000 Adam iterations in one arm, and 1500 Adam iterations followed by L-BFGS in the other (Table 7). The budget does not rescue the problem at this network size. Adam ends at 0.371, 0.343 and 0.380 and is still descending slowly. The L-BFGS phase stops on its own termination test after 8310, 9224 and 7370 iterations, at 0.376, 0.337 and 0.381, with a loss of 0.020 to 0.027, which is 0.16 to 0.19 of its value at the switch and far from zero; Rathore et al. [48] describe the same behaviour of L-BFGS, which in their runs stops before the iteration limit without reaching a critical point. In these six runs the slow mode is accurate to 0.4 to 1.9 per cent, the amplitude of the fast mode is in error by 71 to 83 per cent, and the fast mode carries 89 to 94 per cent of the squared error: the fast oscillation is followed over its first period and then dies out, although the exact amplitude is constant (Figure [S7](#)).

The statement supported by our runs is therefore narrow: with three hidden layers of 64 units, 1024 residual points, unit weights and single precision, neither 41 000 Adam iterations nor an L-BFGS phase run to its termination resolves the fast mode. The runs of Rathore et al. [48] differ from ours in the width of the network (up to 400), the number of residual points (ten times as many), the memory of L-BFGS (100 against 50), a tuned Adam rate and switch point, the initialisation and the selection of the best run; which of these matters was not tested, nor was double precision. None of the remedies proposed in the literature was tried either (Section [8.4](#)).

5.4 Finite-difference baselines and cost

The finite-difference baselines of Section [3.8](#) were run on sequences of grids and evaluated on the held-out sets used for the PINNs. Table 8 lists, for each problem, the coarsest grid that is at least as accurate as the best PINN geometric mean: $n = 100$ for **Heat** and **Lap2**, and the coarsest or second-coarsest grid tried for the other three. The accuracies are not equal: the grid listed has between 0.11 and 0.47 times the error of the PINN, so the comparison is at equal or better accuracy of the finite-difference solve. On **Wave1** the coarsest grid is eight times more accurate

Table 8: PINN against second-order finite differences (FD). “PINN ε_2 ”: lowest geometric mean over seeds among the five strategies, Adam $\rightarrow$ L-BFGS arm, 3000 iterations. FD columns: the coarsest grid of the refinement sequence whose ε_2 on the same held-out set is at most that value (n intervals per space direction), its errors and its solve time (best of three). t_{PINN} : time of one Adam $\rightarrow$ L-BFGS run in the three measurements A, B and C described under “Cost” in the text; ranges are over runs (B) or stacks (C). Ratio: $t_{\text{PINN}}/t_{\text{FD}}$ for B and for C.

	PINN ε_2 (strategy)	n	finite differences			t_{PINN} (s)			ratio	
			ε_2	ε_∞	t_{FD} (s)	(A)	(B)	(C)	B	C
Heat	1.07×10^{-3} (RAD)	100	5.05×10^{-4}	2.71×10^{-4}	2.2×10^{-3}	216	20–37	12	$(0.9\text{--}1.7) \times 10^4$	5.6×10^3
Lap2	2.28×10^{-4} (grid-c)	100	7.51×10^{-5}	6.22×10^{-5}	2.4×10^{-2}	402	46–83	12–14	$(1.9\text{--}3.4) \times 10^3$	$(4.8\text{--}5.7) \times 10^2$
Wave1	0.435 (grid-c)	25	5.14×10^{-2}	9.45×10^{-2}	4.4×10^{-4}	302	60	7	1.3×10^5	1.5×10^4
Wave2	4.61×10^{-2} (Sobol)	20	5.12×10^{-3}	8.83×10^{-3}	3.5×10^{-4}	273	36	7	1.0×10^5	2.1×10^4
Lap3	3.55×10^{-2} (Sobol)	40	9.97×10^{-3}	4.34×10^{-2}	1.6×10^{-3}	341	50	12	3.1×10^4	7.3×10^3

than any benchmark run (6.6 times more accurate than the best of the longer runs), and on Wave2 even the long runs of Section 5.1 (5.7×10^{-3} to 9.3×10^{-3}) stay above the 5.12×10^{-3} of the $n = 20$ grid. The observed order of convergence is 1.99 to 2.01 in all 18 refinements on Heat, Wave1, Wave2 and Lap2; on Lap3 it is 1.97, 2.70 and 1.89, and with the compatible datum of Lap3s it is 1.99, 2.00 and 2.00 (Table S17 and Supplementary Section S4.3).

Cost. The finite-difference solves of Table 8 take 3.5×10^{-4} to 2.4×10^{-2} s. For the PINN we have three measurements, all on the same shared 12-core machine and none on an idle one; they differ in the driver as well as in the load (Supplementary Section S4.4). (A) Single networks on the CPU, timed for 400 Adam and 200 L-BFGS iterations under heavy load and extrapolated to 3000 iterations: 216 to 402 s. (B) Single networks on the CPU, the runs of V-bench at a load average of about 20 to 45: 20 to 83 s. (C) The stacked GPU driver that produced the errors of the benchmark, per network of a stack of 25 or 50: 6.9 to 13.9 s; this amortised time is reached only when that many networks are trained at once. Measurement A is 5.1 to 8.3 times the median of B, and the median of B 2.1 to 8.7 times C, so only orders of magnitude are meaningful. One PINN run costs 1.9×10^3 to 1.3×10^5 times the finite-difference solve that is at least as accurate when a single network is trained (B): three to five orders of magnitude. Per network of a GPU stack (C) the ratio is 4.8×10^2 to 2.1×10^4 , and with A it is 1.7×10^4 to 7.7×10^5 , which is inflated by the load. The smallest ratio in any measurement is 4.8×10^2 (Lap2, C) (Figure S8). The finite-difference times are the best of three solves, the PINN times single measurements; no timing is taken from the longer runs or the control experiments.

This is the expected outcome for linear problems on boxes in one to three dimensions and agrees with the systematic comparison of Grossmann et al. [19] with finite elements. It also illustrates why a classical baseline belongs in every such study [35]. What it does not address are the settings usually put forward for PINNs—irregular geometry, inverse problems, high dimension—of which only the last is examined in this article (Section 6).

6 Dimension scaling: PINN versus Deep Ritz for $d = 2$ to 20

Neural solvers are usually motivated by problems in many dimensions, where a mesh is out of reach [12, 20, 53]. This section measures how the accuracy and the cost of the PINN and of the Deep Ritz method change with the dimension, on the Laplace problem LapD (5) and the Poisson problem PoiD (6) for $d = 2, 3, 5, 10, 20$, with the protocol of Section 3.5: one network ($d\text{--}64\text{--}64\text{--}64\text{--}1$, tanh), 1024 interior and 1024 boundary points redrawn at every iteration, Adam with two learning-rate reductions, 4000 iterations unless stated, $\lambda = 1000$ for the PINN, $\beta = 100$ (LapD) and $\beta = 1$ (PoiD) for Deep Ritz, and the error of the final network on 50 000 held-out points. Each cell has three seeds (0, 1, 2). Within a seed the two methods start from the same

weights and see the same points, so the comparison is paired. One fact frames the results: PoiD has $\partial_n u^* = 0$ and is the best case for the penalised energy, which is then unbiased, whereas LapD is the generic, biased case (Proposition 2); the PINN loss has no such bias (Remark 3). Sections 6.1 to 6.6 report the comparison. Section 6.7 then proves an explicit stability constant for the unit cube, which turns the two terms of the PINN loss into a computable bound on the error of any network, and Section 6.8 tests two cheap remedies for the plateau that both methods reach on PoiD at $d = 20$.

6.1 Accuracy at equal iterations

For every $d \leq 10$ the PINN is the more accurate method on both problems (Tables S3 and S4, Figure 2): the ratio of the mean errors lies between 1.46 and 4.29, and in each of the eight cells the three PINN seeds are below the three Deep Ritz seeds. The H^1 -seminorm errors are ordered in the same way. At $d = 20$ the two methods are level on LapD (ratio 0.99, overlapping ranges) and Deep Ritz is 12 per cent lower on PoiD. In every cell the relative L^2 error on 20 000 held-out boundary points is between 0.65 and 2.11 times the interior error. The verification code V-dim confirms the ordering: at $d = 2$ and $d = 10$, on both problems, each of its PINN seeds is below each of its Deep Ritz seeds (Table S5). It also shows that three seeds understate the spread (on LapD at $d = 20$ its five Deep Ritz seeds have a standard deviation of 4.1×10^{-3} against 1.8×10^{-3} for our three), so we rely only on differences that hold for every seed.

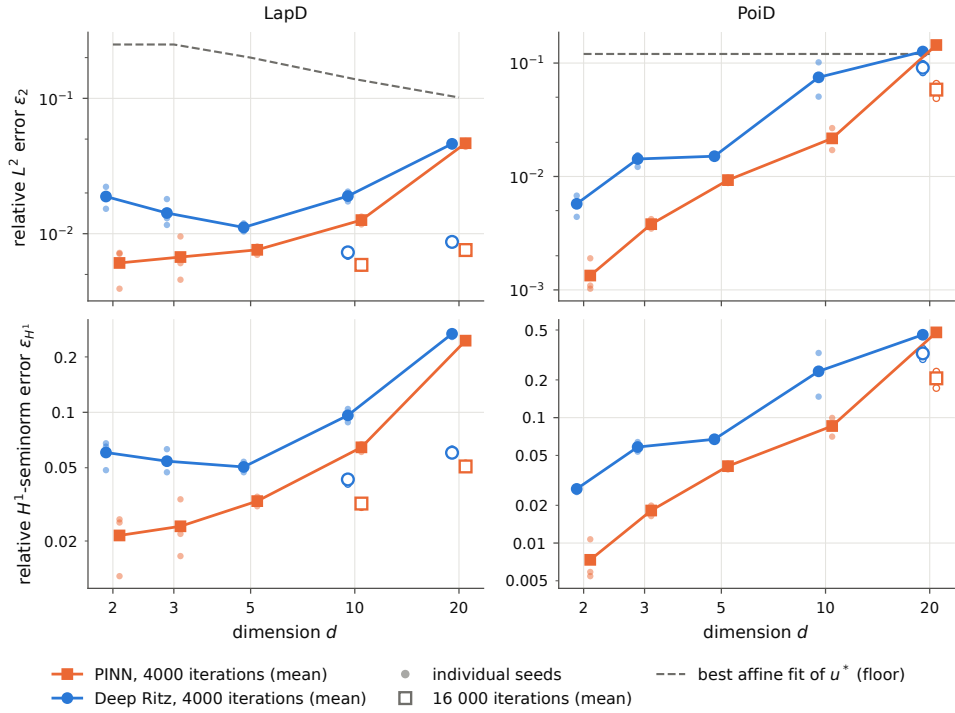


Figure 2: Held-out errors against the dimension for LapD (left) and PoiD (right): relative L^2 error ε_2 (top) and relative H^1 -seminorm error ε_{H^1} (bottom) on 50 000 test points. Filled symbols joined by lines: mean over seeds 0–2 after 4000 iterations (the values of Tables S3 and S4); small dots: the individual seeds. Open symbols: the same after 16 000 iterations, run for LapD at $d = 10$ and for both problems at $d = 20$. Dashed line: ε_2 of the best affine approximation of u^* (Proposition 1). PINN in orange (squares), Deep Ritz in blue (circles); the two methods are offset horizontally for legibility.

6.2 Growth with the dimension, and trivial-predictor floors

At this fixed budget the error grows with d for both methods, except Deep Ritz on LapD for $d \leq 10$, where it is not monotone (1.88×10^{-2} , 1.42×10^{-2} , 1.11×10^{-2} and 1.90×10^{-2} at $d = 2, 3, 5$ and 10). A least-squares line through the logarithm of the geometric-mean error against $\log d$ over $d = 2, \dots, 10$ has slope 0.48 (PINN) and 0.01 (Deep Ritz) on LapD, and 1.71 and 1.46 on PoiD; with $d = 20$ included the slopes are 0.86, 0.41, 1.92 and 1.35. These numbers describe one budget and four or five dimensions. They are not rates: Section 6.3 shows that they change with the budget. Relative errors must be read against the floors of Proposition 1. The solution of LapD concentrates around its mean as d grows, so both floors fall with d and plain ε_2 flatters LapD in high dimension: the centred error of the PINN rises from 9.2×10^{-3} at $d = 2$ to 3.4×10^{-2} at $d = 10$ and 1.73×10^{-1} at $d = 20$, so the “4.7 per cent” of Table S3 at $d = 20$ is 17 per cent of the fluctuation of u^* . For PoiD the affine floor is 0.1203 in every dimension, and at $d = 20$ both methods end above it (0.144 and 0.126): after 4000 iterations neither improves on the best affine function.

6.3 The networks at $d = 20$, and the effect of the budget

The two methods are above the affine floor on PoiD at $d = 20$ for different reasons. Split a function into its least-squares affine fit and a remainder. For the Deep Ritz networks the remainder has 0.18 to 0.20 times the norm of the remainder of u^* , and the residual $\|\Delta u_\theta + f\|/\|f\|$ is 0.90: these networks are still essentially affine. The PINN networks are not. Their residual is 0.35 to 0.37, and their remainder has 0.83 to 0.84 times the right norm, but its correlation with the remainder of u^* is only 0.14 to 0.16: the PINN has built curvature of the right size and the wrong shape. The following elementary bound says where its error sits.

Remark 4. Let $v \in C^2(\overline{\Omega})$, $\Omega = (0, 1)^d$, $e = v - u^*$ and $r = \Delta v + f = \Delta e$. Let $e_0 \in H_0^1(\Omega)$ be the weak solution of $\Delta e_0 = r$; then $e - e_0$ is harmonic and has the boundary values of e . Expanding e_0 and r in the orthonormal Dirichlet eigenfunctions $\prod_i \sqrt{2} \sin(k_i \pi x_i)$ of $-\Delta$, whose eigenvalues $\pi^2 \sum_i k_i^2$ are at least $d\pi^2$, gives $\|e_0\|_{L^2(\Omega)} \leq \|r\|_{L^2(\Omega)}/(d\pi^2)$. For PoiD, where $\|f\| = \pi^2 \|u^*\|$, this reads $\|e_0\|/\|u^*\| \leq d^{-1} \|\Delta v + f\|/\|f\|$.

With the measured residuals, the part of the error that the interior residual accounts for is at most $0.37/20 < 0.02$ of $\|u^*\|$ for the PINN and $0.90/20 = 0.045$ for Deep Ritz, out of total errors of 0.144 and 0.126. For both methods the error is thus mostly a harmonic function fixed by the mismatch on the boundary, which is 16 per cent (PINN) and 13 per cent (Deep Ritz) in relative L^2 norm. Corollary 7(c) completes this remark with an explicit constant for the harmonic part.

The validation curves (Figure S9) show that these errors are limited by the budget. Within 250 iterations the error falls to about the affine floor; it then stalls before descending, and the stall lengthens with d ; at $d = 20$ neither method has left the plateau on PoiD when the 4000-iteration

Table 9: Effect of the budget: ε_2 in units of 10^{-2} , mean $\pm$ sample standard deviation over seeds 0–2. “16 000” is the same protocol with four times as many iterations (learning-rate reductions at 8000 and 12 000). “Equal compute” is Deep Ritz with the number of iterations given in brackets, chosen to match the compute of the 4000-iteration PINN run (Section 6.5). Last column: affine floor. A dash means not run.

	d	4000 iterations		16 000 iterations		equal compute	floor
		PINN	Deep Ritz	PINN	Deep Ritz	Deep Ritz	
LapD	10	1.260 ± 0.084	1.896 ± 0.159	0.588 ± 0.027	0.728 ± 0.039	0.934 ± 0.047 (10 000)	13.9
PoiD	10	2.168 ± 0.482	7.488 ± 2.553	–	–	1.376 ± 0.111 (9000)	12.0
LapD	20	4.663 ± 0.248	4.613 ± 0.178	0.757 ± 0.023	0.870 ± 0.038	–	10.2
PoiD	20	14.424 ± 0.176	12.633 ± 0.022	5.832 ± 0.858	9.102 ± 0.808	–	12.0

run ends. That networks fit the smoothest component first is a known tendency [46]; we cite it as context, not as an explanation. With four times the budget (Table 9) the errors at $d = 10$ on LapD fall by a factor 2.1 (PINN) and 2.6 (Deep Ritz), and at $d = 20$ on LapD by a factor 6.2 and 5.3. On PoiD at $d = 20$ both methods leave the plateau and all six runs end below the affine floor: 5.83×10^{-2} for the PINN and 9.10×10^{-2} for Deep Ritz. These runs are not converged either: the error is still falling when the learning rate is reduced. With the larger budget the PINN is again the more accurate method in every seed at $d = 20$ (ratio 1.15 on LapD, 1.56 on PoiD), so the level or reversed ranking after 4000 iterations was an effect of the budget and, at least in part, of the uncentred input: with the input centred, the PINN is the more accurate method on LapD at $d = 20$ after 4000 iterations as well, in every seed (Section 6.8). Much of the growth with d was also an effect of the budget: from $d = 10$ to $d = 20$ the PINN error on LapD grows by a factor 3.7 after 4000 iterations and 1.3 after 16 000 (Deep Ritz: 2.4 and 1.2). For LapD, Table S3 is therefore to be read as “the number of iterations needed for a given accuracy grows with d ”. For PoiD the corresponding long run at $d = 10$ was not made, and for neither problem was the approximation power of the network tested separately.

6.4 Cost per iteration

Deep Ritz needs only ∇u_θ , one reverse pass whatever the dimension. The PINN needs Δu_θ , that is d second derivatives, either by one further reverse pass per coordinate or by the forward propagation of Section 3.5, which evaluates the same loss. Two separate measurements on the same shared machine (Table S6, Figure S10) agree in the following. From $d = 2$ to $d = 100$ the cost of a Deep Ritz iteration grows by a factor 1.98 or 1.55, from 3.3 to 5.1 ms in the quieter measurement, and that of a PINN iteration roughly linearly, as is known [22]: by a factor 13.9 or 12.7 (forward Laplacian) and 32.5 or 26.2 (nested). At $d = 100$ a PINN iteration costs about 11 to 13 times a Deep Ritz iteration with the forward Laplacian and about 29 to 32 times with the nested one; for $d \geq 10$ the forward Laplacian is 1.8 to 2.9 times cheaper than the nested one. The milliseconds themselves depend on the load and are not portable.

6.5 Accuracy at equal compute

Because a Deep Ritz iteration is cheaper, a comparison at equal iterations understates the method. Deep Ritz was therefore also run with the compute of the 4000-iteration PINN run at $d = 10$. The number of iterations was fixed before the runs, as the mean CPU time of the three PINN runs divided by the mean CPU time per iteration of the three Deep Ritz runs, rounded to a thousand: 10 000 on LapD and 9000 on PoiD, that is 2.5 and 2.25 Deep Ritz iterations for one PINN iteration (the cost ratio measured at $d = 10$ is 2.23 and 2.48 in the two measurements). The ranking reverses (Table 9): Deep Ritz reaches $9.34 \times 10^{-3} \pm 4.7 \times 10^{-4}$ against $1.26 \times 10^{-2} \pm 8.4 \times 10^{-4}$ for the PINN on LapD, and $1.38 \times 10^{-2} \pm 1.1 \times 10^{-3}$ against $2.17 \times 10^{-2} \pm 4.8 \times 10^{-3}$ on PoiD, with disjoint seed ranges in both cases. V-dim agrees (seeds 7–9: 8.74×10^{-3} , 9.09×10^{-3} , 9.02×10^{-3} on LapD; 1.38×10^{-2} , 1.24×10^{-2} , 1.50×10^{-2} on PoiD). In the re-check of Section 6.7, with a separately written trainer and three seeds, Deep Ritz at the CPU time of the PINN was also the more accurate on LapD at $d = 20$ (descriptive). The ranking of the two methods thus depends on whether iterations or compute are counted. It does not show that Deep Ritz is the better method: at 16 000 iterations each the PINN is ahead at $d = 10$ on LapD and at $d = 20$ on both problems, although on LapD that lead is of the size of the penalty bias of Deep Ritz (Section 6.6).

6.6 Sensitivity to the penalty weights, and the penalty bias

The weights were selected at $d = 5$ from one run per value (Table S7). On LapD the error of Deep Ritz falls by a factor 16 from $\beta = 1$ to $\beta = 100$ and is level beyond; the computed penalty

bias at $d = 5$ explains the first part (it is 0.164 for $\beta = 1$ and 2.64×10^{-2} for $\beta = 10$, and these two runs end at 0.98 and 0.88 times it), while for $\beta = 100$ and 1000 the runs end 3.4 and 36 times above the bias, at an error limited by the optimisation. On PoiD, Deep Ritz is flat, as expected from $\partial_n u^* = 0$, and the PINN needs $\lambda \geq 10$. These are statements about $d = 5$. At $d = 10$ larger weights than the chosen ones do not help: on PoiD (V-dim, seeds 7–9) Deep Ritz with $\beta = 10$ or 100 and the PINN with $\lambda = 10^4$ stay on the affine plateau for the whole run (Supplementary Section S5.5).

The penalty bias of Deep Ritz on LapD was computed for every d of the study by a series in the Robin eigenfunctions and, exactly, up to $d = 100$ by the representation of Proposition S11; the two agree to 4.3×10^{-4} relative (Supplementary Section S5.8 and Table S11). At $\beta = 100$ the relative bias $\|u_\beta - u^*\|/\|u^*\|$ falls from 5.5×10^{-3} at $d = 2$ to 7.4×10^{-4} at $d = 20$, roughly like $0.013/d$ to $0.015/d$ for $d \geq 3$. At $d = 2$ it is about the PINN's whole error (6.1×10^{-3}) and 0.29 of the Deep Ritz error. After 4000 iterations it is 27 to 33 per cent of the Deep Ritz error for $d \leq 5$ and 8 and 2 per cent at $d = 10$ and 20, so at that budget the gap between the methods is mostly optimisation. After 16 000 iterations it is 1.05 and 0.65 times the gap to the PINN at $d = 10$ and 20: the lead of the PINN on LapD at the larger budget is of the size of the bias that the fixed $\beta = 100$ imposes on Deep Ritz, and it does not show that the PINN optimises its loss better.

The representation of Proposition S11, a one-dimensional integral over products of one-dimensional Robin eigenfunction sums, is valid for every d and β . Against these exact values, the bound of Corollary 7(b) of the next section is 1.71 to 4.43 times the exact bias at $\beta = 100$ for $d = 2, \dots, 100$, against 6.5 to 49.6 times for the bound of Proposition 2(c). The numbers suggest that $d c(d) \rightarrow \sqrt{8/3}$ for $c(d) = \lim_{\beta \rightarrow \infty} \beta \|u_\beta - u^*\|/\|u^*\|$ (Conjecture S4).

6.7 An explicit stability constant and a computable error bound

Remark 4 bounds the part of the error that the interior residual accounts for; the rest is harmonic and has the boundary values of $v - g$. Its size is governed by

$$\kappa_d = \sup \left\{ \frac{\|\partial_n \psi\|_\partial}{\|\Delta \psi\|} : \psi \in H^2(\Omega) \cap H_0^1(\Omega), \psi \neq 0 \right\}, \quad \Omega = (0, 1)^d, \quad (11)$$

where $\|\cdot\|_\partial$ is the norm of $L^2(\partial\Omega)$ with surface measure ($|\partial\Omega| = 2d$). By Green's formula, $-\int_\Omega w \Delta \psi = -\int_{\partial\Omega} w \partial_n \psi$ for harmonic w and $\psi = 0$ on $\partial\Omega$, so κ_d is the norm of the harmonic extension from $L^2(\partial\Omega)$ to $L^2(\Omega)$ (Supplementary Section S7.5). This is a classical quantity: $\kappa_d^2 = 1/\delta_1$, where δ_1 is the first eigenvalue of the biharmonic Steklov problem $\Delta^2 u = 0$ in Ω , $u = 0$ and $\Delta u = \delta_1 \partial_n u$ on $\partial\Omega$ [5, 6, 16] (Proposition 6). For convex domains of minimal width ℓ , Payne's bound $\delta_1 \geq 2/\ell$ [44], as stated in Bucur and Gazzola [5, Lemma 5.3] (see also [45]), gives $\kappa_d^2 \leq \frac{1}{2}$ on the unit cube in every dimension. The following theorem gives the exact order in d .

Theorem 5 (The harmonic-extension constant of the unit cube). *Let κ_d be defined by (11). For $d \geq 1$,*

$$\max \left\{ L_d, \frac{1}{2d} \right\} \leq \kappa_d^2 \leq U_d,$$

where

$$U_d = \frac{\tanh(\pi\sqrt{d-1}/2)}{\pi\sqrt{d-1}} \quad (d \geq 2), \quad U_1 = \frac{1}{2}, \quad L_d = \frac{4}{\pi^2} \sum_{j \text{ odd}} \frac{j^2}{(j^2 + d - 1)^2}.$$

Moreover $\kappa_1^2 = \frac{1}{2}$, and $U_d < \frac{1}{2}$ for $d \geq 2$. As $d \rightarrow \infty$, $\sqrt{d} U_d \rightarrow 1/\pi$ and $\sqrt{d} L_d \rightarrow 1/(2\pi)$, so κ_d^2 is of exact order $d^{-1/2}$.

Proposition 6 (Biharmonic Steklov eigenvalue). *Let $\Omega = (0, 1)^d$ and $\delta_1 = \inf \{ \|\Delta u\|^2 / \|\partial_n u\|_\partial^2 : u \in H^2(\Omega) \cap H_0^1(\Omega), \partial_n u \neq 0 \}$, the variational form of the first eigenvalue of the biharmonic*

Steklov problem $\Delta^2 u = 0$ in Ω , $u = 0$ and $\Delta u = \delta_1 \partial_n u$ on $\partial\Omega$ [5, (2.2)]. Then $\kappa_d^2 = 1/\delta_1$; hence $1/U_d \leq \delta_1 \leq \min\{1/L_d, 2d\}$.

Corollary 7 (Harmonic part, penalty bias and a computable error bound). *Let $\Omega = (0, 1)^d$ and let $u^* \in C^2(\overline{\Omega})$ solve $-\Delta u^* = f$ in Ω , $u^* = g$ on $\partial\Omega$.*

- (a) *Every $w \in H^1(\Omega)$ with $\int_{\Omega} \nabla w \cdot \nabla \varphi \, d\mathbf{x} = 0$ for all $\varphi \in H_0^1(\Omega)$ satisfies $\|w\| \leq \kappa_d \|w\|_{\partial} \leq \sqrt{U_d} \|w\|_{\partial}$.*
- (b) *The minimiser u_{β} of the penalised Ritz energy (8) satisfies $\|u_{\beta} - u^*\| \leq \sqrt{U_d} \|\partial_n u^*\|_{\partial} / (2\beta)$ for every $\beta > 0$.*
- (c) *For every $v \in C^2(\overline{\Omega})$, in particular every network with tanh activations,*

$$\|v - u^*\| \leq \underbrace{\frac{\|\Delta v + f\|}{d\pi^2}}_{B_{\text{int}}} + \underbrace{\sqrt{U_d} \|v - g\|_{\partial}}_{B_{\text{bd}}} = \frac{\sqrt{\mathcal{L}_{\text{int}}^{\text{pop}}(v)}}{d\pi^2} + \sqrt{2dU_d} \sqrt{\mathcal{L}_{\text{bd}}^{\text{pop}}(v)},$$

where $\mathcal{L}_{\text{int}}^{\text{pop}}(v) = \|\Delta v + f\|^2$ and $\mathcal{L}_{\text{bd}}^{\text{pop}}(v) = \|v - g\|_{\partial}^2 / |\partial\Omega|$ are the mean squared interior residual and the mean squared boundary misfit over the uniform measures, the two terms of the population PINN loss $\mathcal{L}_1^{\text{pop}}$ of Remark 3.

The proofs are in Supplementary Section S7.5; every step was also checked numerically (finite differences on the faces, the partial-fraction identity, Rayleigh quotients of the extremal sequences, and the bound on 200 harmonic test functions per dimension). Corollary 7(c) is Remark 4 completed by an explicit constant for the harmonic part; it needs neither u^* nor a mesh, only the two quantities that the PINN loss estimates. The norm $\|\cdot\|_{\partial}$ uses the surface measure, so in terms of the root-mean-square boundary misfit, the quantity that the boundary term of the loss estimates, the factor is $\sqrt{2dU_d}$, which grows like $d^{1/4}$ (1.08 at $d = 2$, 1.71 at $d = 20$, 2.53 at $d = 100$; Table S9): in high dimension the boundary misfit weighs more in the bound, not less. Part (b) replaces the constant $2C_d$ of Proposition 2(c), which lies between 2.00 and 2.10 for all d , by $\sqrt{U_d}$, which is 0.540, 0.326, 0.270 and 0.179 at $d = 2, 10, 20$ and 100.

Comparison with the classical bounds. Most of that improvement is classical: Payne’s bound already gives the constant $\sqrt{1/2}$, a factor 2.83 to 2.97 below $2C_d$ for $d = 1, \dots, 100$. Against Payne’s constant, Theorem 5 gains the factor $\sqrt{(1/2)/U_d}$, which is 1 at $d = 1$, 1.31 at $d = 2$, 2.17 at $d = 10$ and 3.95 at $d = 100$. The elementary bound through the torsion function, $\kappa_d^2 \leq m_d$ with m_d the largest normal derivative of the torsion function of the cube on its boundary (Remark S9), already decreases with d , but slowly: $m_d = 0.338, 0.195, 0.170$ and 0.137 at $d = 2, 10, 20$ and 100. Against it the theorem gains $\sqrt{m_d/U_d} = 1.08, 1.36, 1.53$ and 2.07 (Table S9). What the theorem adds is therefore the exact order $d^{-1/2}$ of κ_d^2 , not a large constant at small d . The elementary lower bound $1/(2d)$, from the constant harmonic function, is larger than L_d for $2 \leq d \leq 8$; L_d is what gives the order $d^{-1/2}$ from below. How sharp U_d is remains open: the computed Rayleigh quotients fall from $0.985 U_d$ at $d = 2$ to $0.874 U_d$ at $d = 10$ and $0.637 U_d$ at $d = 100$, but the subspaces that can be afforded shrink with d , so these numbers neither support nor refute $\kappa_d^2/U_d \rightarrow 1$; what is proved is $\frac{1}{2}(1 + o(1)) \leq \kappa_d^2/U_d \leq 1$. For the unit square see Supplementary Section S5.7.

The bound on the trained networks. Corollary 7(c) was evaluated on all 88 saved networks of this section, with norms estimated by Monte Carlo on 2×10^5 new interior and 2×10^5 new boundary points per d and, as a second set, on the held-out test points. Three hypotheses were pre-registered and frozen before the confirmatory evaluation: H1, the bound exceeds the error of every network; H2, its efficiency $\eta = (B_{\text{int}} + B_{\text{bd}}) / \|v - u^*\|$ is at most 3 for every confirmatory network with $d \geq 5$; H3, the seed-mean share $B_{\text{bd}} / (B_{\text{int}} + B_{\text{bd}})$ does not decrease with d . Seven of the 88 networks had been seen in a pilot evaluation and are treated as non-confirmatory

(Supplementary Section S5.6). H1 holds: on both sets and for all 88 networks the bound is above the error, by at least 75 standard errors, and the smallest efficiency is 1.744. H2 fails: η exceeds 3 for 21 of the 57 confirmatory networks with $d \geq 5$, with a maximum of 5.46 (Deep Ritz on PoiD at $d = 5$); below $d = 5$, where H2 made no claim, η is up to 8.31 at $d = 3$ and up to 24.8 at $d = 2$. The bound is therefore a valid but loose certificate at small d , not an error estimator. H3 holds for all four (problem, method) pairs: the seed-mean share rises from 0.19 to 0.29 at $d = 2$ (0.056 for Deep Ritz on PoiD) to 0.84 to 0.98 at $d = 20$ (Figure 3). Part of this trend is built into the constants, $1/(d\pi^2)$ for the interior and $\sqrt{2dU_d} \sim d^{1/4}$ for the boundary, so H3 alone does not show that the error itself is dominated by the boundary misfit; at $d = 20$ Remark 4 does show it for the networks tested, since B_{int} is at most 0.36 of the error on every main network. The re-check repeated the evaluation with its own evaluator (η within -0.78 to $+0.61$ per cent on all 88 networks, the same decisions) and retrained 45 networks with separately written trainers and other seeds, on LapD, PoiD and two problems outside the study; this part was not pre-registered. Over all 133 networks evaluated the bound held in every case; its efficiency is 1.92 to 3.44 at $d = 10$ and 20 and up to 24.8 at $d = 2$. At $d = 20$ and equal CPU time the bound ranks all 3×3 pairs of a retrained PINN and Deep Ritz network as the error does; at equal iterations, where the errors of the two methods lie within the seed spread, it ranks only 4 of the 9 pairs correctly: a bound within a factor of about 2 of the error is not meant to resolve error differences much smaller than that factor.

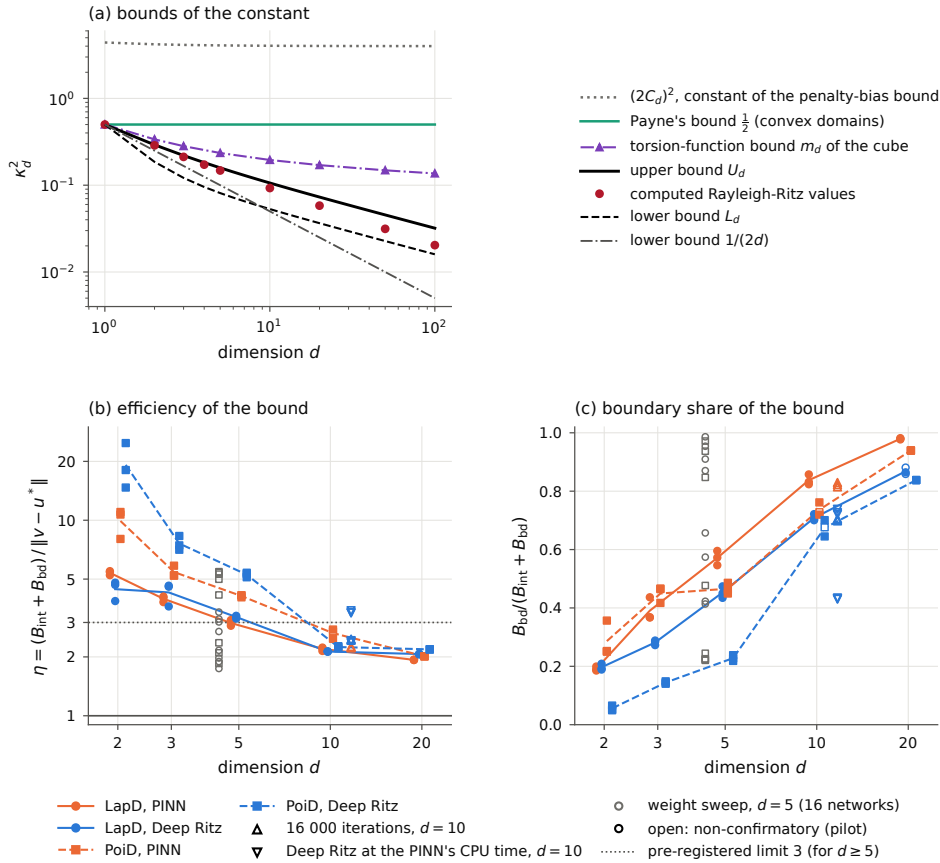


Figure 3: (a) The bounds U_d , L_d and $1/(2d)$ of Theorem 5 on κ_d^2 , Payne's bound $\frac{1}{2}$, the torsion-function bound m_d of the cube (Remark S9), the largest computed Rayleigh quotients, and the square of the constant $2C_d$ of Proposition 2(c). (b) Efficiency η and (c) boundary share of the bound of Corollary 7(c) for all 88 saved networks, new Monte-Carlo set: the main runs (lines: seed means; open markers: the seven networks seen in the pilot), the 16 000-iteration and equal-compute runs at $d = 10$ (triangles), and the 16 networks of the weight sweep at $d = 5$ (grey, drawn slightly to the left of $d = 5$). The dotted line in (b) is the pre-registered limit 3.

Relation to the literature. Besides the classical facts recalled above (Fichera’s duality, Payne’s bound, the torsion function), numerical bounds for δ_1 on the square are in Kuttler [27] and Ferrero et al. [16]. Theorem 5 adds a two-sided bound of κ_d^2 of the exact order $d^{-1/2}$ on the hypercube (equivalently, κ_d is of exact order $d^{-1/4}$), with an elementary proof (sine series, the Cauchy–Schwarz inequality, partial fractions). In the literature searched for this work no bound of κ_d^2 of the order $d^{-1/2}$ was found. A posteriori bounds for network approximations exist in other norms and settings: Ernst et al. [13] certify in the energy norm with constants not explicit in d ; Führer and Rojas [17] measure the boundary misfit in $H^{1/2}(\partial\Omega)$ with mesh-dependent constants; Zeinhofer et al. [64] work on $C^{1,1}$ domains with non-explicit constants; in the generalisation bounds of Mishra and Molinaro [36] the boundary misfit enters through its square root with constants that contain norms of the network; and the bound $\max|v - u^*| \leq \max_{\partial\Omega}|v - g| + \frac{1}{8} \max|\Delta v + f|$ from the maximum principle, explicit on the cube [38, Prop. 7], needs suprema, which the Monte-Carlo losses do not estimate (on our networks its sampled value was 5.7 to 42 times the bound of Corollary 7(c)). The closest form is in the error analysis of residual minimisation by Shin et al. [51, App. B, eq. (B2)]: $\|u\| \leq C(\|\Delta u\| + \|u\|_{\partial})$, from the regularity theory of Bramble and Schatz [4], linear in the L^2 norms of the residual and of the boundary misfit as in Corollary 7(c), but on smooth domains and with a constant that is not explicit. With Payne’s constant a bound of this form follows at once from the duality, so Corollary 7(c) is an application of known theory with a better constant, not a new method. Imposing $v = g$ exactly [56] would remove B_{bd} and the penalty bias; it was not tested here. For the penalty bias, $O(1/\beta)$ is known on $C^{1,1}$ domains [39], and error bounds with extrapolation exist for the penalty finite-element method [24]; Proposition S11 is a routine separation of variables whose value lies in the numbers it gives on the cube.

6.8 Two cheap remedies at $d = 20$

At $d = 20$ and 4000 iterations neither method improves on the best affine approximation of PoiD (Section 6.2); this is what is called the *plateau* below: a test error no smaller than the affine floor. Two cheap remedies suggest themselves, and neither is new. The first, called *presolve* here, writes the approximation as a fixed known part plus a network, as in the trial solutions of Lagaris et al. [28], with a cheap Galerkin solution as the known part, as in Galerkin neural networks [1] and multi-level networks [2]: the fixed part is the exact minimiser of the method’s own population loss over affine functions, obtained from one $(d + 1) \times (d + 1)$ linear system (Proposition S14). The second, called *lift3c*, replaces the input $\mathbf{x}$ by the coordinate-wise Legendre features $P_1(t_i), P_2(t_i), P_3(t_i)$, $t_i = 2x_i - 1$, so that the first layer has $3d$ inputs. Polynomial input layers trained with a PINN loss are known, for high-frequency solutions: HOrderDNN [7], whose first layer is a tensor-product polynomial basis, and its coordinate-wise successor K-HOrderDNN [65], tested up to $d = 10$ for PDEs and $d = 50$ for function fitting; related are the first layers of Chebyshev-basis Kolmogorov–Arnold networks [37, 52] and feature-engineered or Fourier-feature PINNs [15, 59]. Mapping the input to $[-1, 1]^d$ is common practice (the reference code of Raissi et al. [47] does so), and the centred-input network $\mathcal{N}(2\mathbf{x} - 1)$, called *lift1*, is reported as a baseline. That low-degree or low-frequency components are learned first, which motivates both remedies, is known [18, 46, 59]. What is measured here is whether either remedy removes the plateau on PoiD, and how each changes the accuracy at $d = 20$ on further problems, for both methods, at equal iterations and at equal CPU time.

Design. The test was pre-registered: hypotheses, problems, seeds 10 to 14, budgets, metric and decision rule were fixed in a document whose hash and time were recorded in the repository before the first counted run (no external registry was used). Both remedies had been tried on LapD and PoiD at $d = 20$ in exploratory pilots with seeds 0 to 2 before the freeze, as had the centred input (on LapD at $d = 20$, on PoiD up to $d = 16$) and, for Deep Ritz on LapD, rep3, so the direction of their effects on these two problems was known. Only two problems

were held out, on which neither remedy had been tried before the test (a related ridge had been used in an earlier pilot): the ridge RidgeD, $u^* = \cos(2s)$ with $s = d^{-1/2} \sum_i (2x_i - 1)$, and CosPairD, $u^* = \sum_{k \leq d/2} \cos(\pi x_{2k-1}) \cos(\pi x_{2k})$. On PoiD the lift features contain the solution up to 3.9×10^{-3} , so PoiD is reported but excluded from the decision on the lift. The metric is the relative L^2 error on the held-out test points, and the effect of an arm is the seed-paired ratio $\rho = \varepsilon_2(\text{plain})/\varepsilon_2(\text{arm})$ ($\rho > 1$: the arm is more accurate). By the frozen rule a remedy *helps* on a problem if, for both methods, $\rho > 1$ in at least 4 of 5 seeds with median $\rho \geq 1.5$, at 4000 iterations and at equal CPU time; it *hurts* if for one method a median ρ is at most $1/1.1$; it is a *general remedy* if it helps on LapD and on RidgeD or CosPairD and hurts on none of them. Two controls were pre-registered, for Deep Ritz only: *lift3u*, which spans the same functions as lift3c but uses the uncentred feature x_i in place of $P_1(t_i)$, and *rep3*, the featureless input (t, t, t) with $t = 2\mathbf{x} - 1$; the centred input lift1 was added for both methods as an exploratory arm after the frozen analysis. Three problems were added afterwards: under an addendum to the pre-registration, the ridge AltRidgeD, $u^* = \cos(\pi s' + 1)$ with $s' = d^{-1/2} \sum_i (-1)^{i+1} (2x_i - 1)$ and $f = 4\pi^2 u^*$; and, in the descriptive second round of the re-check, SinLinD, $u^* = \sum_{k \leq d/2} \sin(\pi x_{2k-1}) x_{2k}$ with $f = \pi^2 u^*$, and ExpCosD, $u^* = \sum_{k \leq d/2} e^{x_{2k-1}} \cos x_{2k}$ with $f = 0$. The full design, with the weights, the equal-CPU budgets and the later additions, is in Supplementary Section S5.9.

Results. Presolve helps on LapD only (median ratios 2.67 for Deep Ritz and 2.85 for the PINN, 5 of 5 seeds each; Table S12). On PoiD it is neutral by the frozen rule (median 1.00 for Deep Ritz) and for the PINN slightly worse in every seed (median 0.94, range 0.91 to 0.98); it changes nothing on CosPairD, where the affine minimiser is zero (Proposition S15), and hurts on the held-out ridge RidgeD (medians 0.66 and 0.79, no seed better than plain). On the four pre-registered problems lift3c is more accurate than plain in 5 of 5 seeds for both methods, at 4000 iterations (median ratios 2.33 and 2.86 on LapD, 2.27 and 1.77 on RidgeD, 2.83 and 4.02 on CosPairD, for Deep Ritz and the PINN) and at equal CPU time, so by the frozen rule it is a general remedy on LapD, RidgeD and CosPairD. The controls do not attribute this gain to the Legendre features on every problem (Tables S19 and S13(b)). On LapD lift3u is less accurate than plain in every seed (median 0.52), which triggered the pre-registered centring clause, so no statement on the features is made there; no control reproduces the gain (rep3 0.88, 0 of 5 seeds better; lift1 0.92 for Deep Ritz, 2 of 5), and lift3c is 2.43 (Deep Ritz) and 1.61 (PINN) times more accurate than lift1, 5 of 5 seeds each. On RidgeD centring the input alone matches the whole gain for Deep Ritz, and the PINN is more accurate with lift1 than with lift3c in every seed (median 0.77). On CosPairD lift3c also beats lift1 (2.23 and 2.39, 5 of 5), but for Deep Ritz the featureless reparametrisation rep3 gains about as much as lift3c there, as on PoiD and RidgeD. AltRidgeD was run with the study’s own code under the addendum, frozen before these runs; it is not a blind test, since three seeds of the re-check had already shown the sign of the lift effect on it. There the lift hurts (Deep Ritz median ratio 0.59 at 4000 iterations and 0.32 at equal CPU time, 0 of 5 seeds better; PINN 0.88 and 0.72), and so does presolve (Deep Ritz 0.73, PINN 0.83), whereas the centred input alone makes the PINN more accurate (1.29, 5 of 5). By the extended rule of the addendum neither presolve nor lift3c is a general remedy on the five problems. In the second round of the re-check (Deep Ritz, three seeds, a different implementation, descriptive) the gain of the lift on SinLinD is again matched by centring the input, and ExpCosD is on the plateau ($\varepsilon_2 = 0.025$, affine floor 0.022), which presolve leaves (median ratio 4.51).

What this shows and what it does not. At $d = 20$, with this network, optimiser, schedule and these weights, the accuracy of both methods depends strongly on how the input enters the first layer, in a way that differs between problems; no single mechanism (centring the input or the polynomial features) accounts for it on every problem. Of the five problems of the study only PoiD is on the plateau after 4000 iterations (plain $\varepsilon_2 = 0.126$ for Deep Ritz and 0.145 for the PINN, against the affine floor 0.119 on the test set), and of the two of the re-check only

ExpCosD; on the others plain already ends below the affine floor, so there the comparison is one of accuracy at a fixed budget, not of escaping a plateau. On PoiD presolve does not leave the plateau. lift3c does, for both methods and in every seed. For Deep Ritz the gain cannot be attributed to its features, although they contain the solution of PoiD up to 3.9×10^{-3} : the featureless control rep3 is more accurate still (median ratio 8.10 against 6.59) and lift3u gains as much (6.71). The centred input alone (lift1, exploratory) already ends below the affine floor in every seed for both methods ($\varepsilon_2 = 0.070$ to 0.091 for Deep Ritz and 0.067 to 0.080 for the PINN, against 0.119), but with less gain than lift3c, which is 4.22 times (Deep Ritz) and 7.51 times (PINN) more accurate than lift1 in median, 5 of 5 seeds each (Table S13(b)); for the PINN, for which no featureless control was run, the features thus add to centring on PoiD. At this budget the plateau of PoiD is therefore not robust to centring the input, which is common practice. As a remedy the result is negative: neither remedy is general, and neither is new as a method. What is added is a measurement of both, for the PINN and Deep Ritz side by side, at equal iterations and at equal CPU time, on two problems held out from the pilots and three added later, with two pre-registered controls for Deep Ritz (lift3u, rep3) and an exploratory centred-input arm for both methods.

6.9 Comparison with published results

For LapD at $d = 10$, E and Yu [12] report a relative L^2 error of about 0.4 per cent after 50 000 iterations with a 671-parameter network and $\beta = 10^3$. Our mean errors at $d = 10$ after 16 000 iterations are 0.59 per cent (PINN) and 0.73 per cent (Deep Ritz), with best seeds at 0.56 and 0.68 per cent, for a different network, sampling and schedule. The orders of magnitude agree; we do not claim to reproduce their number. For the Neumann example whose solution PoiD shares, they report 1.3 per cent at $d = 5$ and 1.9 per cent at $d = 10$ with the Ritz method and no boundary penalty; equation, boundary condition, network and budget differ from ours, so this is not a like-for-like comparison with Table S4. Chen et al. [8] set the deep Galerkin method [53], whose loss is the residual least-squares loss of a PINN, against the Deep Ritz method on elliptic problems with boundary conditions imposed by penalty. They report that Deep Ritz can outperform the residual method even for smooth solutions, with a clear dependence on the dimension, that the residual method can outperform Deep Ritz for solutions of low regularity, and that imposing the boundary condition exactly, where that is possible, gives a better solution and an easier training. Our finding that the PINN is the more accurate method per iteration for $d \leq 10$ on two smooth problems is therefore not a general ranking, and it does not contradict theirs: networks, sampling, penalty weights and budgets differ, and our own ranking changes with what is counted and with the budget. What the two studies have in common is that neither loss dominates the other.

7 Two implementation pitfalls, demonstrated on minimal examples

A PINN can reach a small training loss and still be far from the solution for reasons that have nothing to do with the optimiser or the points. This section demonstrates two such reasons on minimal, released examples. Neither depends on seeds or budgets, and each is caught by a cheap test that we recommend in Section 8.2.

7.1 A residual that automatic differentiation makes identically zero

A tempting way to obtain u_{tt} in PyTorch is to differentiate u_t with respect to the column $\mathbf{x}[:, 0]$ of the input tensor $\mathbf{x}$. Indexing creates a new tensor, and u_t was computed from $\mathbf{x}$, not from that slice, so reverse-mode differentiation [43] finds no path between the two. Without

`allow_unused=True` the call raises an error; with it the call returns `None`, and code that replaces a missing derivative by zeros turns the residual into the constant zero, which gives the loss neither a value nor a gradient. The correct pattern differentiates with respect to the whole input and indexes the result (Listing S1). A unit test exposes the fault at once: in double precision on 2000 random points the faulty operator for the wave equation $u_{tt} = \Delta u$ returns exactly 0 both for the solution of `Wave2` and for the non-solution $e^{-t} \sin \pi x \sin \pi y$, whereas the correct operator returns at most 3.6×10^{-15} for the former and values up to 19.6 for the latter.

Training gives no warning. We trained a 3–50–50–1 tanh network on the domain of `Wave2` with a loss made of the faulty residual, a data term that prescribes $e^{-t} \sin \pi x \sin \pi y$ at 10 000 random space–time points and a zero boundary term at 400 points of the spatial boundary at $t = 0$ (batches of 64, Adam with learning rate 10^{-3} , seed 0, 23 550 iterations). The PDE term was exactly 0 at every iteration (all 70 650 second-derivative calls returned `None`), while the loss fell from 0.10 to 1.1×10^{-4} . The network fitted the data term (relative distance 0.025 on the held-out grid of `Wave2`) and followed its decay, with mode amplitudes 0.981 and 0.364 at $t = 0$ and 1; its relative distance from the solution of `Wave2` is 1.35. The network thus follows whatever the data term prescribes; a correct residual would have penalised this data term, which is not a wave:

Proposition 8 (A decaying mode is not a wave solution). *Let $\Omega = (-1, 1)^2$, $S(x, y) = \sin(\pi x) \sin(\pi y)$ and $\omega = \sqrt{2} \pi$.*

- (a) *The function $v(t, x, y) = e^{-t} S(x, y)$ satisfies $v_{tt} - \Delta v = (1 + 2\pi^2) e^{-t} S$, which vanishes nowhere in $(0, \infty) \times \{S \neq 0\}$; v is not a solution of the wave equation $u_{tt} = \Delta u$.*
- (b) *The unique solution of `Wave2` is $u^* = \cos(\omega t) S$, and the relative distance between v and u^* in $L^2((0, 1) \times \Omega)$ is*

$$\frac{\|v - u^*\|}{\|u^*\|} = \left(\frac{\int_0^1 (e^{-t} - \cos \omega t)^2 dt}{\int_0^1 \cos^2 \omega t dt} \right)^{1/2} \approx 1.380.$$

On the held-out grid, which has 21 time levels, the ratio of part (b) is 1.353. With ten times as many iterations nothing changes in kind: the PDE term is exactly 0 at all 235 500 iterations, the loss falls to 5.6×10^{-6} , and the distance from the solution is 1.35. Three guards would each catch the fault: never pass `allow_unused=True` in a residual; assert that every derivative is not `None`; and unit-test each residual operator on an exact solution *and* on a non-solution. The last is decisive, since a test on an exact solution alone is passed by the zero operator.

7.2 A small loss does not imply a unique solution

A boundary-value problem with too few conditions has a whole family of solutions, and a PINN trained on it may converge to any of them. The Laplace equation in the cube with data on one face is the simplest example.

Proposition 9 (A Laplace problem with data on one face only). *Let $D = (0, \pi)^3$. The boundary-value problem*

$$\Delta u = 0 \quad \text{in } D, \quad u(\pi, y, z) = \sin y \cos z \quad \text{for } (y, z) \in [0, \pi]^2,$$

with no condition on the other five faces, has infinitely many solutions in $C^\infty(\overline{D})$. Precisely:

- (a) $u_0(x, y, z) = \cosh(\sqrt{2}(x - \pi)) \sin y \cos z$ is a solution;
- (b) if u is a solution and h is a harmonic polynomial in the two variables (y, z) , then $u + (x - \pi) h(y, z)$ is a solution; the solutions therefore form an affine space of infinite dimension;
- (c) for every $A \in \mathbb{R}$ the solution $u_0 + A(x - \pi)$ takes the value $-A\pi$ at the centre $(0, \pi/2, \pi/2)$ of the face $\{x = 0\}$, so solutions exist whose values on the unconstrained faces are arbitrarily large;

- (d) the function $u_N(x, y, z) = \sin y \cos z \sinh(\sqrt{2}x)/\sinh(\sqrt{2}\pi)$, which vanishes on $\{x = 0\}$, $\{y = 0\}$, $\{y = \pi\}$ and has zero normal derivative on $\{z = 0\}$ and $\{z = \pi\}$, is another solution, different from u_0 .

We trained five seeded 2×100 tanh networks on this problem, with the residual on the closed 21^3 grid of the cube and the datum on its 441 points with $x = \pi$ (AdamW [33] with learning rate 10^{-3} and its default weight decay, 2000 iterations). Both loss terms end between 2.7×10^{-4} and 2.2×10^{-3} in every run, and the five networks are five different, nearly harmonic functions: their relative L^2 distance from the solution of Lap3, which has the same datum and zero data on the other five faces, is 1.62, 12.51, 13.22, 7.28 and 1.63, and the mean of $|u_\theta|$ on the face $x = 0$ is 0.27, 2.62, 2.65, 1.45 and 0.26 (Figure S13), in line with part (c) of the proposition. A re-run of seeds 0 to 2 with the separately written code V-bench (Section 3.9) gave the distances 1.59, 12.84 and 13.36. Nothing in such a loss selects one solution, so the outcome depends on the initial weights. Count the conditions before training and say why the solution is unique; and run more than one seed: an answer that changes with the seed while the loss stays comparably small is the symptom.

The settings of the two demonstrations are arbitrary, were not tuned and differ from the protocol of Section 3. The mechanism of each, a residual that is identically zero and a loss that does not determine the solution, does not depend on them; the particular numbers do.

8 Discussion, limitations and conclusion

8.1 What the evidence supports

Findings (1) to (5) are ordered by the strength of their evidence, strongest first; (6) and (7) concern Sections 6.7 and 6.8, and (8) the reporting of errors.

- (1) *Two implementation faults give a small loss and a wrong answer.* A residual made identically zero by a misused automatic-differentiation call, and a boundary-value problem with data on one face of six, each gave a small training loss and a wrong answer (Section 7): in the first the network follows whatever the data term prescribes, which a correct residual would have penalised. Neither fault depends on seeds or budgets, and each is exposed by a cheap test.
- (2) *An L-BFGS phase is the most effective change tested.* At equal iteration count it lowered the held-out error in all 175 pairs (39 of 40 in the verification code), by about one order of magnitude on Heat and Lap2 even against the best logged Adam iterate, and a longer phase took Wave2 below one per cent (Sections 4.2 and 5.1). The Adam arm was not tuned.
- (3) *The choice among space-filling point sets is secondary.* No strategy is best on every problem; the largest effect after the L-BFGS phase, a fourfold penalty of the cell-centred grid on Heat, is caused by the absence of residual points next to the initial line (Table 6). Adaptive sampling brought no detectable gain at our budget, and redrawing the points at every iteration hurt under Adam alone (on Lap2 by a factor 6.4, the largest penalty of any strategy in the benchmark) but not after the L-BFGS phase (Section 4.3).
- (4) *At the sizes and budgets used, the unit-weight PINN does not solve the two-mode wave problem, and finite differences are far faster.* All 50 runs on Wave1 ended between 41 and 51 per cent error and the longer runs between 34 and 38 per cent, although Rathore et al. [48] report a few per cent with wider networks, more points and a tuned schedule. A second-order finite-difference solve was at least as accurate as the best PINN on all five problems and faster by a factor of more than 400 in every timing (smallest measured ratio 4.8×10^2 ; Section 5.4), as Grossmann et al. [19] found against finite elements.
- (5) *In higher dimension the ranking of PINN and Deep Ritz depends on what is counted.* Per iteration the PINN was the more accurate for $d \leq 10$ and, with 16 000 iterations, at $d = 20$; at equal compute Deep Ritz was the more accurate (at $d = 10$, and at $d = 20$ for LapD in a descriptive re-check with three seeds; Sections 6.1–6.5). On LapD the lead of the PINN at

the larger budget is of the size of the penalty bias of Deep Ritz (Section 6.6). As in Chen et al. [8], neither loss dominates.

- (6) *On the unit cube the error of a network is bounded by the two population terms of its loss, but the bound is loose at small d .* The constant of the harmonic extension is of exact order $d^{-1/4}$ (Theorem 5); the bound of Corollary 7 held on all 133 networks evaluated, with efficiency 1.92 to 3.44 at $d = 10$ and 20, but the pre-registered target of an efficiency of at most 3 for $d \geq 5$ failed (21 of 57 networks), and at $d = 2$ the efficiency reached 24.8. It is a certificate, not an error estimator. A bound of this form already follows from Payne’s classical constant; the theorem adds the order in d .
- (7) *Neither cheap remedy is general at $d = 20$.* By the pre-registered rule the Legendre features (lift3c) were a general remedy on the four pre-registered problems, but they and the fixed affine part were less accurate than plain on AltRidgeD, which was added afterwards and was not a blind test; the affine part also hurt on RidgeD. The features left the plateau of PoiD, whose solution they contain up to 3.9×10^{-3} , as did the centred input alone (exploratory, with less gain) and, for Deep Ritz, the featureless control rep3 (with more); how much of each gain the features explain differs between problems (Section 6.8).
- (8) *How an error is reported changes the conclusion.* A relative L^2 error needs the error of a trivial predictor beside it (Proposition 1), and for a decaying solution the space–time error hides the late slices: all 40 runs of Section 4.5 are more than 100 per cent in error on the slice $t = 1$, at a space–time error of about 0.1 per cent after Adam $\rightarrow$ L-BFGS.

8.2 A checklist for small PINN studies

1. Unit-test every residual operator on an exact solution and on a non-solution, and never let a missing derivative default to zero (Section 7.1).
2. Count the initial and boundary conditions and say why the solution is unique (Section 7.2).
3. Fix and report seeds, at least five, preferably ten, per configuration, and compare paired by seed.
4. Report held-out relative errors, not training losses, per time slice for evolution problems (Section 4.5), with the error of a trivial predictor beside them (Proposition 1).
5. Run a classical baseline on the same evaluation set, after checking it against an exact solution (Section 5.4).

8.3 Limitations

Budgets. The benchmark uses 256 to 1024 interior points and 3000 iterations. The strategy effects belong to this scarce-point regime.

Statistics. Wave1, Wave2 and Lap3 have five seeds, for which the paired test cannot go below $p = 0.0625$, so “no significant difference” is weak evidence there. The p -values are not corrected for multiple comparisons, and the rule by which we call an effect significant (all ten seeds agree; Section 3.7) was fixed after the results were known.

Verification. The re-checks were made with separately written code within the same project, on the same machine and library versions; they are not a replication by a third party. The longer runs, the control experiments, the time-slice result, the 16 000-iteration runs at $d = 20$ and the demonstration of Section 7.1 rest on the code of the experiments alone (Section 3.9).

Scope. All problems are linear with constant coefficients on boxes, and their solutions are smooth except at the discontinuous data of Lap3. Each problem has one tanh architecture, and Sections 4 and 5 use unit loss weights. Uniqueness is proved in a stated class (classical solutions; for Lap3, the solutions that attain the datum in mean square); continuous dependence on the data is not proved.

Unfinished problems. Wave2 and Lap3 are not converged at 3000 iterations. The longer runs on Wave2 (three seeds, one strategy) ended on the tolerance of the optimiser in single precision,

below one per cent but not on a plateau (Section 5.1). Wave1 is not solved at the network size and budgets used (Section 5.3).

Implementation. Most multi-seed runs used single precision on a GPU backend. A stack repeated on the same machine reproduces its results bit for bit, but the result of one network depends on the stack in which it is trained (its size and composition; the mechanism, presumably rounding, was not investigated; Supplementary Section S8.4); other hardware was not tested. The L-BFGS of these runs is a custom stacked implementation whose final errors were 4 to 10 per cent above those of the reference optimiser, in geometric mean, in the validation of Supplementary Section S8.2.

Timings come from three measurements on a loaded shared machine, with different drivers, and are good to an order of magnitude only.

Dimension study. There are three seeds per cell, which understates the spread (Table S5) and allows no test, and no network was trained for $d > 20$. One architecture and one learning-rate schedule serve every d ; the schedule itself arrests the descent, so the errors at a fixed budget include its effect. The weights were tuned at $d = 5$ with one run per value, and λ and β are normalised differently in d (Remark 3). Both solutions are simple in structure, a sum of products of pairs of coordinates and a sum of functions of one coordinate, and that of PoiD, with zero normal derivative, favours Deep Ritz. Boundary conditions are imposed by penalty only. The inputs of the dimension study are not centred; centring alone (exploratory, Section 6.8) changes the errors at $d = 20$ and 4000 iterations by median factors of 0.92 to 1.95 (0.83 to 2.27 per seed), makes the PINN more accurate than Deep Ritz on LapD and takes both methods below the affine floor on PoiD. The 16 000-iteration runs exist for $d = 10$ on LapD and for $d = 20$ only, and the equal-compute comparison for $d = 10$ and, in the re-check, for LapD at $d = 20$ with three seeds.

Stability constant and remedies. Theorem 5 and the bound of Corollary 7 are proved for the unit cube only, and whether U_d is sharp is open; the bound holds for the exact norms and was evaluated with Monte-Carlo estimates of them, with margins of at least 75 standard errors on the 88 saved networks and 92 on the 21 networks of the second round of the re-check; for the 24 networks of its first round (efficiency at least 1.97) no standard errors were recorded. The remedies were tested with the network, optimiser, schedule and weights of Section 6, with five seeds at $d = 20$ and three at $d = 5$ and 10, and with CPU-time budgets measured on a loaded shared machine. Both remedies, the centred input and, for Deep Ritz on LapD, rep3 had been tried on LapD and PoiD in exploratory pilots before the freeze, so their direction on these two problems was known and only RidgeD and CosPairD were held out; the pre-registered controls lift3u and rep3 were run for Deep Ritz only; AltRidgeD was not a blind test; and the second round of the re-check, which added SinLinD and ExpCosD, is descriptive, and three planned cells of it were not run (Supplementary Section S5.9).

8.4 What was not tried

Apart from the two cheap remedies of Section 6.8, none of the published remedies for the failures seen here was tried: adaptive loss weighting [58, 60], causal training [61], Fourier-feature inputs [59], or trial functions that satisfy the initial and boundary conditions exactly [28, 49, 56]; on the cube the last would also remove the boundary term of the bound of Section 6.7 and the penalty bias. For Deep Ritz we tried neither continuation in β nor a consistent boundary treatment of Nitsche type [41], which exists in a neural version [31]. Non-linear and variable-coefficient equations, domains other than boxes, and more seeds are the natural next steps.

8.5 Conclusion

We posed eight linear problems so that each has a unique solution in a stated class (Proposition S2), measured seeded held-out errors of PINNs and the Deep Ritz method against exact solutions and, in low dimension, finite-difference baselines, and checked the benchmark and the dimension study

in part with separately written code. On these problems an L-BFGS phase mattered most; after it, and once the strip next to the initial line had residual points, the collocation strategy changed the error by at most about a factor of two in geometric mean ($N_r \geq 256$); a classical solver was far faster; a unit-weight PINN of the size used did not solve the two-mode wave problem; and whether the PINN or Deep Ritz is ahead in higher dimension depended on the budget, on what is counted, on the input scaling and, on the Laplace problem, on the penalty bias. On the unit cube a classical stability constant now has explicit two-sided bounds of the right order in d ; with it, as already with Payne’s classical constant, the two population terms of the PINN loss bound the L^2 error of any C^2 network, and the theorem sharpens that bound. The bound held on every network evaluated but is loose. Of two cheap remedies at $d = 20$, the fixed affine part did not remove the plateau of the Poisson problem, and the Legendre features, which contain its solution up to 0.4 per cent, did, as did centring the input alone (exploratory); neither remedy improved the accuracy on every problem. Finally, the two pitfalls of Section 7 show that a small training loss is not evidence of a correct solution; each is caught by an entry of the checklist above.

Data and code availability

All code, raw results, figure scripts and the L^AT_EX sources of this article and of its Supplementary Material are available at

<https://github.com/zhouyi-xiaoxiao/pinn-diffusion-wave-laplace>. Every figure is produced by a script from the stored per-run results in that repository, and every table of results is generated from them by a script or, for the tables assembled in the text, checked against them by one; the file `reproduce.md` gives the script and the exact command for each, and `notes/VERIFICATION.md` describes how the results were checked. The script `code/check_headline_numbers.py` checks the numbers of the abstract against the stored results. All runs are seeded. The analyses of Sections 6.7 and 6.8 are in the folders `research_highdim_bound` and `research_highdim_remedies`, each with its pre-registration documents, scripts, stored results and the scripts and outputs of its re-checks. Trained networks are not included. This applies also to the 88 networks on which the bound of Section 6.7 was evaluated: their SHA-256 hashes, recorded before the evaluation, and the outputs of the evaluation are included, and the training scripts retrain the networks (on the machine used, seven runs of the dimension study were repeated bit for bit), so the evaluation can be repeated only on retrained networks.

Acknowledgements

This article develops a project report written in 2024 during an exchange year at the Hong Kong University of Science and Technology.

References

- [1] Mark Ainsworth and Justin Dong. Galerkin neural networks: a framework for approximating variational equations with error control. *SIAM Journal on Scientific Computing*, 43(4):A2474–A2501, 2021. doi: 10.1137/20M1366587.
- [2] Ziad Aldirany, Régis Cottureau, Marc Laforest, and Serge Prudhomme. Multi-level neural networks for accurate solutions of boundary-value problems. *Computer Methods in Applied Mechanics and Engineering*, 419:116666, 2024. doi: 10.1016/j.cma.2023.116666. arXiv:2308.11503.
- [3] Ivo Babuška. The finite element method with penalty. *Mathematics of Computation*, 27(122):221–228, 1973. doi: 10.1090/S0025-5718-1973-0351118-5.
- [4] James H. Bramble and Alfred H. Schatz. Rayleigh–Ritz–Galerkin methods for Dirichlet’s problem using subspaces without boundary conditions. *Communications on Pure and Applied Mathematics*, 23(4):653–675, 1970. doi: 10.1002/cpa.3160230408.

- [5] Dorin Bucur and Filippo Gazzola. The first biharmonic Steklov eigenvalue: positivity preserving and shape optimization. *Milan Journal of Mathematics*, 79(1):247–258, 2011. doi: 10.1007/s00032-011-0143-x.
- [6] Dorin Bucur, Alberto Ferrero, and Filippo Gazzola. On the first eigenvalue of a fourth order Steklov problem. *Calculus of Variations and Partial Differential Equations*, 35(1):103–131, 2009. doi: 10.1007/s00526-008-0199-9.
- [7] Zhipeng Chang, Ke Li, Xiufen Zou, and Xueshuang Xiang. High order deep neural network for solving high frequency partial differential equations. *Communications in Computational Physics*, 31(2):370–397, 2022. doi: 10.4208/cicp.OA-2021-0092.
- [8] Jingrun Chen, Rui Du, and Keke Wu. A comparison study of deep Galerkin method and deep Ritz method for elliptic problems with different boundary conditions. *Communications in Mathematical Research*, 36(3):354–376, 2020. doi: 10.4208/cmr.2020-0051.
- [9] J. Crank and P. Nicolson. A practical method for numerical evaluation of solutions of partial differential equations of the heat-conduction type. *Proceedings of the Cambridge Philosophical Society*, 43(1):50–67, 1947. doi: 10.1017/S0305004100023197.
- [10] Salvatore Cuomo, Vincenzo Schiano Di Cola, Fabio Giampaolo, Gianluigi Rozza, Maziar Raissi, and Francesco Piccialli. Scientific machine learning through physics-informed neural networks: Where we are and what’s next. *Journal of Scientific Computing*, 92(3):88, 2022. doi: 10.1007/s10915-022-01939-z.
- [11] Arka Daw, Jie Bu, Sifan Wang, Paris Perdikaris, and Anuj Karpatne. Mitigating propagation failures in physics-informed neural networks using retain-resample-release (R3) sampling. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 7264–7302, 2023. URL <https://proceedings.mlr.press/v202/daw23a.html>.
- [12] Weinan E and Bing Yu. The Deep Ritz method: A deep learning-based numerical algorithm for solving variational problems. *Communications in Mathematics and Statistics*, 6(1):1–12, 2018. doi: 10.1007/s40304-018-0127-z.
- [13] Lewin Ernst, Nikolaos Rekasinas, and Karsten Urban. A posteriori certification of PDE approximations with particular application to neural networks, 2025. URL <https://arxiv.org/abs/2502.20336>. Version 6, August 2026.
- [14] Lawrence C. Evans. *Partial Differential Equations*, volume 19 of *Graduate Studies in Mathematics*. American Mathematical Society, Providence, RI, second edition, 2010. doi: 10.1090/gsm/019.
- [15] Shaghayegh Fazliani, Zachary Frangella, and Madeleine Udell. Enhancing physics-informed neural networks through feature engineering. *Transactions on Machine Learning Research*, 2025. URL <https://arxiv.org/abs/2502.07209>.
- [16] Alberto Ferrero, Filippo Gazzola, and Tobias Weth. On a fourth order Steklov eigenvalue problem. *Analysis*, 25(4):315–332, 2005. doi: 10.1524/anly.2005.25.4.315.
- [17] Thomas Führer and Sergio Rojas. A posteriori analysis of neural network approximations, 2025. URL <https://arxiv.org/abs/2507.06017>.
- [18] Nikhil Ghosh, Song Mei, and Bin Yu. The three stages of learning dynamics in high-dimensional kernel methods. In *International Conference on Learning Representations (ICLR)*, 2022. URL <https://arxiv.org/abs/2111.07167>.
- [19] Tamara G. Grossmann, Urszula Julia Komorowska, Jonas Latz, and Carola-Bibiane Schönlieb. Can physics-informed neural networks beat the finite element method? *IMA Journal of Applied Mathematics*, 89(1):143–174, 2024. doi: 10.1093/imamat/hxae011.
- [20] Jiequn Han, Arnulf Jentzen, and Weinan E. Solving high-dimensional partial differential equations using deep learning. *Proceedings of the National Academy of Sciences*, 115(34):8505–8510, 2018. doi: 10.1073/pnas.1718942115.
- [21] Zhongkai Hao, Jiachen Yao, Chang Su, Hang Su, Ziao Wang, Fanzhi Lu, Zeyu Xia, Yichi Zhang, Songming Liu, Lu Lu, and Jun Zhu. PINNacle: A comprehensive benchmark of physics-informed neural networks for solving PDEs. In *Advances in Neural Information Processing Systems*, volume 37, pages 76721–76774, 2024. doi: 10.52202/079017-2442.
- [22] Zheyuan Hu, Khemraj Shukla, George Em Karniadakis, and Kenji Kawaguchi. Tackling the curse of dimensionality with physics-informed neural networks. *Neural Networks*, 176:106369, 2024. doi: 10.1016/j.neunet.2024.106369.
- [23] George Em Karniadakis, Ioannis G. Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu

- Yang. Physics-informed machine learning. *Nature Reviews Physics*, 3(6):422–440, 2021. doi: 10.1038/s42254-021-00314-5.
- [24] J. Thomas King. New error bounds for the penalty method and extrapolation. *Numerische Mathematik*, 23(2):153–165, 1974. doi: 10.1007/BF01459948.
 - [25] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR)*, 2015. URL <https://arxiv.org/abs/1412.6980>.
 - [26] Aditi S. Krishnapriyan, Amir Gholami, Shandian Zhe, Robert M. Kirby, and Michael W. Mahoney. Characterizing possible failure modes in physics-informed neural networks. In *Advances in Neural Information Processing Systems*, volume 34, pages 26548–26560, 2021. URL <https://proceedings.neurips.cc/paper/2021/hash/df438e5206f31600e6ae4af72f2725f1-Abstract.html>.
 - [27] J. R. Kuttler. Remarks on a Stekloff eigenvalue problem. *SIAM Journal on Numerical Analysis*, 9(1):1–5, 1972. doi: 10.1137/0709001.
 - [28] Isaac E. Lagaris, Aristidis Likas, and Dimitrios I. Fotiadis. Artificial neural networks for solving ordinary and partial differential equations. *IEEE Transactions on Neural Networks*, 9(5):987–1000, 1998. doi: 10.1109/72.712178.
 - [29] Randall J. LeVeque. *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. Society for Industrial and Applied Mathematics, Philadelphia, 2007. doi: 10.1137/1.9780898717839.
 - [30] Ruichen Li, Haotian Ye, Du Jiang, Xuelan Wen, Chuwei Wang, Zhe Li, Xiang Li, Di He, Ji Chen, Weiluo Ren, and Liwei Wang. A computational framework for neural network-based variational Monte Carlo with Forward Laplacian. *Nature Machine Intelligence*, 6(2):209–219, 2024. doi: 10.1038/s42256-024-00794-x.
 - [31] Yulei Liao and Pingbing Ming. Deep Nitsche method: Deep Ritz method with essential boundary conditions. *Communications in Computational Physics*, 29(5):1365–1384, 2021. doi: 10.4208/cicp.OA-2020-0219.
 - [32] Dong C. Liu and Jorge Nocedal. On the limited memory BFGS method for large scale optimization. *Mathematical Programming*, 45(1–3):503–528, 1989. doi: 10.1007/BF01589116.
 - [33] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *7th International Conference on Learning Representations (ICLR)*, 2019. URL <https://arxiv.org/abs/1711.05101>.
 - [34] Lu Lu, Xuhui Meng, Zhiping Mao, and George Em Karniadakis. DeepXDE: A deep learning library for solving differential equations. *SIAM Review*, 63(1):208–228, 2021. doi: 10.1137/19M1274067.
 - [35] Nick McGreivy and Ammar Hakim. Weak baselines and reporting biases lead to overoptimism in machine learning for fluid-related partial differential equations. *Nature Machine Intelligence*, 6(10):1256–1269, 2024. doi: 10.1038/s42256-024-00897-5.
 - [36] Siddhartha Mishra and Roberto Molinaro. Estimates on the generalization error of physics-informed neural networks for approximating PDEs. *IMA Journal of Numerical Analysis*, 43(1):1–43, 2023. doi: 10.1093/imanum/drab093. arXiv:2006.16144; ETH Zürich SAM research report 2020-45.
 - [37] Farinaz Mostajeran and Salah A. Faroughi. Scaled-cPIKANs: Spatial variable and residual scaling in Chebyshev-based physics-informed Kolmogorov-Arnold networks. *Journal of Computational Physics*, 537:114116, 2025. doi: 10.1016/j.jcp.2025.114116. arXiv:2501.02762.
 - [38] Amartya Mukherjee, Maxwell Fitzsimmons, David C. Del Rey Fernández, and Jun Liu. Rigorous error certification for neural PDE solvers: From empirical residuals to solution guarantees, 2026. URL <https://arxiv.org/abs/2603.19165>. Version 2, 1 September 2026.
 - [39] Johannes Müller and Marius Zeinhofer. Error estimates for the Deep Ritz method with boundary penalty. In *Proceedings of Mathematical and Scientific Machine Learning*, volume 190 of *Proceedings of Machine Learning Research*, pages 215–230, 2022. URL <https://proceedings.mlr.press/v190/muller22a.html>.
 - [40] Mohammad Amin Nabian, Rini Jasmine Gladstone, and Hadi Meidani. Efficient training of physics-informed neural networks via importance sampling. *Computer-Aided Civil and Infrastructure Engineering*, 36(8):962–977, 2021. doi: 10.1111/mice.12685.
 - [41] J. Nitsche. Über ein Variationsprinzip zur Lösung von Dirichlet-Problemen bei Verwendung von Teilräumen, die keinen Randbedingungen unterworfen sind. *Abhandlungen aus dem Mathematischen Seminar der Universität Hamburg*, 36(1):9–15, 1971. doi: 10.1007/BF02995904.
 - [42] Art B. Owen. Scrambling Sobol’ and Niederreiter–Xing points. *Journal of Complexity*, 14(4):466–489, 1998. doi: 10.1006/jcom.1998.0487.

- [43] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, pages 8024–8035, 2019. URL <https://arxiv.org/abs/1912.01703>.
- [44] L. E. Payne. Bounds for the maximum stress in the Saint Venant torsion problem. *Indian Journal of Mechanics and Mathematics*, pages 51–59, 1968/69. Special issue presented to Professor Bibhutibhusan Sen on his seventieth birthday, part I.
- [45] G. A. Philippin and A. Safoui. On extending some maximum principles to convex domains with nonsmooth boundaries. *Mathematical Methods in the Applied Sciences*, 33(15):1850–1855, 2010. doi: 10.1002/mma.1295.
- [46] Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred A. Hamprecht, Yoshua Bengio, and Aaron Courville. On the spectral bias of neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 5301–5310, 2019. URL <https://proceedings.mlr.press/v97/rahaman19a.html>.
- [47] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019. doi: 10.1016/j.jcp.2018.10.045.
- [48] Pratik Rathore, Weimu Lei, Zachary Frangella, Lu Lu, and Madeleine Udell. Challenges in training PINNs: A loss landscape perspective. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 42159–42191, 2024. URL <https://proceedings.mlr.press/v235/rathore24a.html>.
- [49] Hailong Sheng and Chao Yang. PFNN: A penalty-free neural network method for solving a class of second-order boundary-value problems on complex geometries. *Journal of Computational Physics*, 428:110085, 2021. doi: 10.1016/j.jcp.2020.110085.
- [50] Yeonjong Shin, Jérôme Darbon, and George Em Karniadakis. On the convergence of physics informed neural networks for linear second-order elliptic and parabolic type PDEs. *Communications in Computational Physics*, 28(5):2042–2074, 2020. doi: 10.4208/cicp.OA-2020-0193.
- [51] Yeonjong Shin, Zhongqiang Zhang, and George Em Karniadakis. Error estimates of residual minimization using neural networks for linear PDEs. *Journal of Machine Learning for Modeling and Computing*, 4(4):73–101, 2023. doi: 10.1615/JMachLearnModelComput.2023050411. arXiv:2010.08019.
- [52] Khemraj Shukla, Juan Diego Toscano, Zhicheng Wang, Zongren Zou, and George Em Karniadakis. A comprehensive and FAIR comparison between MLP and KAN representations for differential equations and operator networks. *Computer Methods in Applied Mechanics and Engineering*, 431:117290, 2024. doi: 10.1016/j.cma.2024.117290. arXiv:2406.02917.
- [53] Justin Sirignano and Konstantinos Spiliopoulos. DGM: A deep learning algorithm for solving partial differential equations. *Journal of Computational Physics*, 375:1339–1364, 2018. doi: 10.1016/j.jcp.2018.08.029.
- [54] I. M. Sobol’. On the distribution of points in a cube and the approximate evaluation of integrals. *USSR Computational Mathematics and Mathematical Physics*, 7(4):86–112, 1967. doi: 10.1016/0041-5553(67)90144-9.
- [55] Walter A. Strauss. *Partial Differential Equations: An Introduction*. John Wiley & Sons, Hoboken, NJ, second edition, 2008. ISBN 978-0-470-05456-7.
- [56] N. Sukumar and Ankit Srivastava. Exact imposition of boundary conditions with distance functions in physics-informed deep neural networks. *Computer Methods in Applied Mechanics and Engineering*, 389:114333, 2022. doi: 10.1016/j.cma.2021.114333. arXiv:2104.08426.
- [57] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, 2020. doi: 10.1038/s41592-019-0686-2.
- [58] Sifan Wang, Yujun Teng, and Paris Perdikaris. Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM Journal on Scientific Computing*, 43(5):A3055–A3081, 2021. doi: 10.1137/20M1318043.
- [59] Sifan Wang, Hanwen Wang, and Paris Perdikaris. On the eigenvector bias of Fourier feature networks:

- From regression to solving multi-scale PDEs with physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 384:113938, 2021. doi: 10.1016/j.cma.2021.113938.
- [60] Sifan Wang, Xinling Yu, and Paris Perdikaris. When and why PINNs fail to train: A neural tangent kernel perspective. *Journal of Computational Physics*, 449:110768, 2022. doi: 10.1016/j.jcp.2021.110768.
 - [61] Sifan Wang, Shyam Sankaran, and Paris Perdikaris. Respecting causality for training physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 421:116813, 2024. doi: 10.1016/j.cma.2024.116813.
 - [62] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945. doi: 10.2307/3001968.
 - [63] Chenxi Wu, Min Zhu, Qinyang Tan, Yadhu Kartha, and Lu Lu. A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 403:115671, 2023. doi: 10.1016/j.cma.2022.115671.
 - [64] Marius Zeinhofer, Rami Masri, and Kent-André Mardal. A unified framework for the error analysis of physics-informed neural networks. *IMA Journal of Numerical Analysis*, 45(5):2988–3025, 2025. doi: 10.1093/imanum/drae081. arXiv:2311.00529.
 - [65] Yaqin Zhang, Ke Li, Zhipeng Chang, Xuejiao Liu, Yunqing Huang, and Xueshuang Xiang. A Kolmogorov high order deep neural network for high frequency partial differential equations in high dimensions. *Communications in Computational Physics*, 38(1):181–222, 2025. doi: 10.4208/cicp.OA-2024-0095. arXiv:2502.01938.